REȚELE DE CALCULATOARE · SEMESTRUL 1 · CURSUL 4

Nivelul aplicație II: continuarea Web-ului, accesul securizat la distanță și transferul de fișiere

Pachet didactic pentru săptămâna 4: note de curs, seminar, laborator Linux și laborator Windows (câte două ore fiecare)

Conf. univ. dr. Nicolaie Popescu-Bodorin, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Partea	Conținut	Durată
A	Cursul 4 — Nivelul aplicație II: starea și cache-ul HTTP, HTTP/2 și HTTP/3, HTTPS, TELNET, SSH și FTP	2 h
B	Seminarul 4 — Cache-uri, handshake-uri, tuneluri și două conexiuni	2 h
C	Laboratorul 4 (Linux) — Sesiuni, securitate și transfer de fișiere: cookie-uri, HTTPS, TELNET, SSH și FTP	2 h
D	Laboratorul 4 (Windows) — Aceleași protocoale cu instrumente Windows și Winsock	2 h

Partea A · Cursul 4

După acest curs ar trebui să puteți

• explica modul în care cookie-urile adaugă stare protocolului HTTP, care este fără stare, și ce costă ele în privința confidențialității;
• descrie cache-ul web și GET-ul condiționat și calcula efectul unui cache asupra timpului de răspuns;
• spune ce schimbă HTTP/2 și HTTP/3 în modul în care circulă mesajele HTTP și de ce;
• explica ce adaugă TLS unei conexiuni TCP, număra drumurile dus-întors ale handshake-ului său și spune ce rămâne vizibil;
• descrie TELNET și explica de ce a fost înlocuit de SSH;
• numi cele trei niveluri ale SSH, explica cheile de host și autentificarea cu cheie publică și folosi redirecționarea de porturi;
• descrie FTP: cele două conexiuni ale sale, modul activ și pasiv, comenzile și răspunsurile sale și modul în care organizează datele pe care le transferă.

Planul celor două ore

Partea	Subiect	Timp
Ora 1		
1	Trei săptămâni de nivel aplicație: unde ne aflăm	5 min

Partea	Subiect	Timp
2	Stare peste un protocol fără stare: cookie-urile	12 min
3	Cache-ul web, GET-ul condiționat și distribuția de conținut	18 min
4	HTTP/2 and HTTP/3	15 min
	Pauză	10 min
Ora 2		
5	HTTPS: HTTP peste TLS	15 min
6	Autentificare la distanță în clar: TELNET	8 min
7	SSH: shell securizat, tuneluri securizate	15 min
8	FTP: două conexiuni și un dialog cu stare; rezumat	12 min

1. Trei săptămâni de nivel aplicație: unde ne aflăm

Programa analitică ne cere să acoperim, la nivelul aplicație, protocoalele și serviciile FTP, HTTP, HTTPS, SSH, SMTP, POP, IMAP, TELNET, Usenet și DNS, împreună cu translatarea adreselor de rețea (NAT). Le-am distribuit pe trei săptămâni. Săptămâna 3 a pus bazele — arhitecturi, socket-uri, servicii de transport, Web-ul și HTTP/1.1. Săptămâna aceasta completează HTTP, îl securizează cu TLS și studiază protocoalele pentru lucrul pe o mașină aflată la distanță și pentru transferul de fișiere către și dinspre aceasta. Săptămâna 5 este dedicată poștei electronice, știrilor (news) și infrastructurii de nume și adresare pe care se sprijină toate celelalte protocoale.

Subiect	Săptămâna 3	Săptămâna 4	Săptămâna 5
HTTP (Hypertext Transfer Protocol)	obiecte, URL-uri, mesaje HTTP/1.1, conexiuni persistente, încadrare (framing)	**cookie-uri, cache, GET condiționat, HTTP/2, HTTP/3**	redirecționare CDN cu DNS
HTTPS (Hypertext Transfer Protocol Secure)	TLS menționat ca nivel de securitate	**handshake-ul TLS, certificate, ce rămâne vizibil**	DNS peste HTTPS
TELNET	—	**autentificare la distanță în clar, negocierea opțiunilor**	—
SSH (Secure Shell)	—	**arhitectură, chei, autentificare, tuneluri, scp și SFTP**	—
FTP (File Transfer Protocol)	—	**organizarea datelor, gestionarea conexiunilor, modul activ și pasiv**	FTP prin NAT
SMTP, POP, IMAP (poștă electronică)	—	—	**tratare completă**
Usenet	—	—	**tratare completă**
DNS (Domain Name System)	numele sunt rezolvate înainte de conectare	—	**tratare completă**
NAT (translatarea adreselor de rețea)	nodurile P2P din spatele NAT	de ce nu funcționează FTP-ul activ	**tratare completă**

2. Stare peste un protocol fără stare: cookie-urile

HTTP este **fără stare**: fiecare cerere este înțeleasă în sine, iar serverul nu trebuie să o țină minte pe cea anterioară. Totuși, multe aplicații au nevoie de stare — o autentificare, un coș de cumpărături, o preferință de limbă. Răspunsul Web-ului, standardizat în RFC 6265, este **cookie-ul**: o mică bucată de date pe care serverul îi cere browserului să o stocheze și să o trimită înapoi odată cu cererile ulterioare.

1. Primul răspuns conține un antet precum `Set-Cookie: session=7f3a9c; Path=/; HttpOnly; Secure; SameSite=Lax`.
2. Browserul stochează cookie-ul, împreună cu site-ul de la care a venit.
3. Fiecare cerere ulterioară către acel site îl trimite înapoi: `Cookie: session=7f3a9c`.
4. Serverul folosește valoarea drept cheie în propria bază de date, unde se află starea propriu-zisă: cine sunteți, ce aveți în coș.

Așadar, cele patru ingrediente sunt o linie de antet în răspuns, o linie de antet în cerere, un fișier păstrat de browser și o bază de date păstrată de server. Protocolul însuși rămâne fără stare; starea se află la cele două capete, legate prin valoarea cookie-ului — încă o instanță a argumentului capăt-la-capăt (end-to-end) din Cursul 2.

Atribut	Semnificație
`Expires`, `Max-Age`	cât timp păstrează browserul cookie-ul; fără ele este un **cookie de sesiune**, șters la închiderea browserului
`Domain`, `Path`	către ce host-uri și căi îl trimite înapoi browserul
`Secure`	se trimite doar prin HTTPS, niciodată în clar
`HttpOnly`	invizibil pentru JavaScript-ul din pagină: un script injectat de un atacator nu îl poate citi
`SameSite`	`Strict`, `Lax` sau `None`: dacă cookie-ul însoțește cererile inițiate de *alte* site-uri — o apărare împotriva falsificării cererilor între site-uri (cross-site request forgery)

> **Cookie-urile și confidențialitatea**
> Un cookie setat de site-ul pe care îl vizitați este un cookie **primar (first-party)**. O pagină poate încărca și obiecte — o reclamă, un buton „like”, un script de analiză — de la o terță parte, al cărei răspuns poate seta propriul **cookie terț (third-party)**. Deoarece aceeași terță parte este încorporată în mii de site-uri, cookie-ul ei îi permite să urmărească un browser prin toate acestea și să construiască un profil al utilizatorului. De aceea legislația europeană (Directiva ePrivacy și GDPR) cere consimțământ pentru cookie-urile neesențiale și de aceea browserele restricționează tot mai mult cookie-urile terțe — Safari și Firefox le blochează implicit.

3. Cache-ul web, GET-ul condiționat și distribuția de conținut

Un **cache web** păstrează copii ale obiectelor cerute recent, astfel încât cererile ulterioare să poată fi servite fără a mai merge la serverul de origine. Cache-uri există în mai multe locuri: în interiorul browserului, într-un **proxy** administrat de o instituție sau de un ISP, în **proxy-uri inverse (reverse proxies)** aflate în fața unui server și în serverele unei **rețele de distribuție a conținutului** (CDN). Un proxy partajat este în același timp client și server: primește cererea browserului, răspunde din

memoria proprie dacă poate, iar altfel transmite cererea către origine, păstrează o copie a răspunsului și o transmite mai departe.

Cache-ul ajută în trei moduri: **timpul de răspuns** scade pentru obiectele găsite în cache (un *hit*); **traficul pe legătura de acces** scade, ceea ce poate amâna o modernizare costisitoare; iar **serverul de origine** este mai puțin încărcat. Exemplul rezolvat de mai jos leagă săptămâna aceasta de rezultatele privind cozile de așteptare din Cursul 2.

> **Exemplu rezolvat — ce face un cache unei legături de acces**
> Rețeaua unei instituții (1 Gbit/s) ajunge la Internet printr-o **legătură de acces de 100 Mbit/s**. Utilizatorii cer 95 de obiecte pe secundă; obiectele au în medie **1 Mbit**; un server de origine se află în medie la 1 s distanță (drum dus-întors plus timpul serverului). Intensitatea traficului pe legătura de acces este 95 × 1 / 100 = **0,95**. Cu formula M/M/1 din Cursul 2, întârzierea medie pe legătură este (L/R) / (1 − ρ) = 10 ms / 0,05 = **200 ms**, deci o cerere durează aproximativ 1,2 s — iar cu cinci la sută mai mult trafic ar exploda.
> Instalați un cache cu o **rată de hit de 40 %**. Doar 57 de obiecte pe secundă traversează legătura: ρ = 0,57, întârzierea pe legătură 10 / 0,43 ≈ **23 ms**. Timpul mediu de răspuns ≈ 0,4 × (câteva ms în LAN) + 0,6 × (1 s + 0,023 s) ≈ **0,62 s** — jumătate din timpul de dinainte, fără a cumpăra o legătură mai rapidă.

3.1 Prospețime și validare

Un cache nu trebuie să servească o copie expirată. HTTP (RFC 9111) îi oferă două instrumente. **Prospețimea (freshness)**: serverul spune cât timp poate fi folosită copia fără a întreba, cu `Cache-Control: max-age=3600` (secunde) sau cu data mai veche `Expires`; un cache raportează de cât timp deține o copie în antetul `Age`. **Validarea**: când o copie nu mai este proaspătă, cache-ul întreabă originea dacă s-a schimbat, printr-un **GET condiționat**.

```
GET /files/notes.pdf HTTP/1.1                    HTTP/1.1 304 Not Modified
Host: cn.example.edu                             ETag: "5f3a-1e2b4"
If-None-Match: "5f3a-1e2b4"                     Cache-Control: max-age=3600
If-Modified-Since: Fri, 16 Oct 2026 14:12:30 GMT
```

Validatorii sunt `ETag` — un identificator de versiune opac pe care serverul l-a atașat obiectului — și data `Last-Modified`. Dacă obiectul nu s-a schimbat, serverul răspunde **304 Not Modified** fără corp; cache-ul reîmprospătează durata de viață a copiei și o servește. Dacă s-a schimbat, serverul răspunde 200 cu noul obiect. O cerere mică și un răspuns mic înlocuiesc un transfer complet: acesta este GET-ul condiționat întâlnit în Seminarul 3, Problema 4.

Directivă	Semnificație
`max-age=N`	proaspăt timp de N secunde
`no-cache`	poate fi stocat, dar trebuie **revalidat** înainte de fiecare utilizare
`no-store`	nu trebuie stocat deloc (pagini bancare, date personale)
`private` / `public`	doar browserul îl poate păstra în cache / îl pot păstra și cache-urile partajate
`s-maxage=N`	o durată de viață separată pentru cache-urile partajate, precum CDN-urile

3.2 Rețele de distribuție a conținutului

Un **CDN** — Akamai, Cloudflare sau rețelele private ale Google, Netflix și Meta — operează mii de servere de cache în interiorul sau în vecinătatea ISP-urilor de acces și în punctele de schimb de trafic Internet (IXP). Obiectele populare sunt copiate aproape de utilizatori, astfel încât majoritatea cererilor parcurg câteva milisecunde în loc să traverseze continente. Rămân două întrebări pentru mai târziu: cum este direcționată cererea unui utilizator către un server CDN apropiat — de obicei prin **DNS**, care răspunde la același nume cu adrese diferite în locuri diferite (Săptămâna 5) — și cum este transmis video în flux (streaming) de la astfel de servere, în bucăți adaptate debitului disponibil.

4. HTTP/2 and HTTP/3

HTTP/1.1 trimite pe o conexiune un mesaj după altul. Un răspuns mare sau lent întârzie tot ce se află la coadă în spatele lui — **blocarea la capul cozii (head-of-line blocking)** — iar browserele ocolesc problema deschizând șase conexiuni paralele către fiecare server, fiecare cu propriul handshake și propriul slow start. HTTP/2 (2015, acum RFC 9113) și HTTP/3 (2022, RFC 9114) păstrează **semantica** HTTP — metode, coduri de stare, antete, URL-uri — și schimbă doar modul în care sunt transportate mesajele.

4.1 HTTP/2: o conexiune, multe fluxuri

- **Încadrare binară.** Mesajele sunt tăiate în **cadre (frames)**, fiecare cu un antet de 9 octeți care indică lungimea, tipul și **identificatorul de flux**. Cadrele HEADERS transportă câmpurile de antet, cadrele DATA corpul mesajului.
- **Multiplexare.** Fiecare schimb cerere–răspuns este un **flux (stream)**; cadrele mai multor fluxuri sunt întrețesute pe o singură conexiune TCP. O imagine mare nu mai reține un script mic: expeditorul le întrețese cadrele și poate acorda priorități.
- **Compresia antetelor (HPACK).** Câmpurile de antet se repetă de la o cerere la alta (cookie-uri, user agent); HPACK trimite un index într-un tabel al câmpurilor trimise anterior în locul textului.
- **Server push**, care permitea unui server să trimită obiecte înainte ca acestea să fie cerute, s-a dovedit greu de folosit bine și este acum dezactivat de browserele majore.

HTTP/2 elimină blocarea la capul cozii la nivelul HTTP, dar nu și la nivelul TCP: TCP livrează un flux de octeți în ordine, astfel încât un singur segment pierdut oprește **toate** fluxurile până când este retransmis. Pe legăturile mobile și Wi-Fi cu pierderi, o singură conexiune HTTP/2 poate fi mai lentă decât șase conexiuni HTTP/1.1.

4.2 HTTP/3: HTTP peste QUIC

QUIC (RFC 9000, 2021) este un protocol de transport construit peste UDP și implementat în interiorul aplicațiilor. Oferă **fluxuri independente** fiabile și cu control al congestiei — o pierdere oprește doar fluxul căruia îi aparține — și integrează TLS 1.3, astfel încât o conexiune nouă are nevoie de un singur drum dus-întors înainte de cerere, iar una reluată își poate trimite cererea imediat (**0-RTT**). Conexiunile QUIC sunt identificate prin **ID-uri de conexiune** și nu prin 4-tuplu, astfel încât un telefon care trece de pe Wi-Fi pe rețeaua mobilă își păstrează conexiunea. HTTP/3 înseamnă HTTP peste QUIC, cu QPACK în locul HPACK. Un browser află că un server îl suportă dintr-un antet de răspuns `Alt-Svc` sau dintr-o înregistrare DNS, iar astăzi o mare parte din traficul web — în special către Google, Meta și Cloudflare — folosește HTTP/3.

	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP (+ TLS)	TCP + TLS	QUIC peste UDP (cu TLS 1.3 inclus)
Formatul mesajelor	text	cadre binare	cadre binare
Concurență	conexiuni paralele	fluxuri pe o singură conexiune	fluxuri independente
Blocare la capul cozii	la nivelul HTTP și TCP	doar la nivelul TCP	niciuna între fluxuri
Compresia antetelor	niciuna	HPACK	QPACK
Drumuri dus-întors înainte de prima cerere (conexiune nouă)	1 (+1–2 pentru TLS)	2 cu TLS 1.3	1

5. HTTPS: HTTP peste TLS

HTTPS înseamnă HTTP trimis prin **TLS** (Transport Layer Security, versiunea 1.3 în RFC 8446) pe portul 443. TLS se situează între aplicație și TCP: aplicația scrie octeți simpli într-o bibliotecă TLS, care îi criptează și scrie înregistrări (records) în socket. Oferă trei garanții:

- **confidențialitate** — un observator nu poate citi datele;
- **integritate** — orice modificare a datelor în tranzit este detectată;
- **autentificarea serverului** — clientul știe că vorbește cu proprietarul numelui pe care l-a cerut, nu cu un impostor (autentificarea clientului este opțională).

5.1 Handshake-ul

Înainte să circule datele, clientul și serverul convin asupra algoritmilor și cheilor. În TLS 1.3, primul mesaj al clientului (**ClientHello**) conține deja suitele de cifruri suportate și o **parte de cheie (key share)** — jumătatea sa dintr-un schimb Diffie–Hellman pe curbe eliptice. Serverul răspunde cu propria parte de cheie (**ServerHello**), **certificatul** său, o semnătură care dovedește că deține cheia privată a certificatului și un mesaj **Finished**; din acel moment ambele părți calculează aceleași chei de sesiune, pe care niciun observator nu le poate calcula. Clientul verifică certificatul, trimite propriul Finished și, în același zbor (flight), prima sa cerere HTTP. TLS 1.2 avea nevoie de încă un drum dus-întors; QUIC combină handshake-urile de transport și TLS.

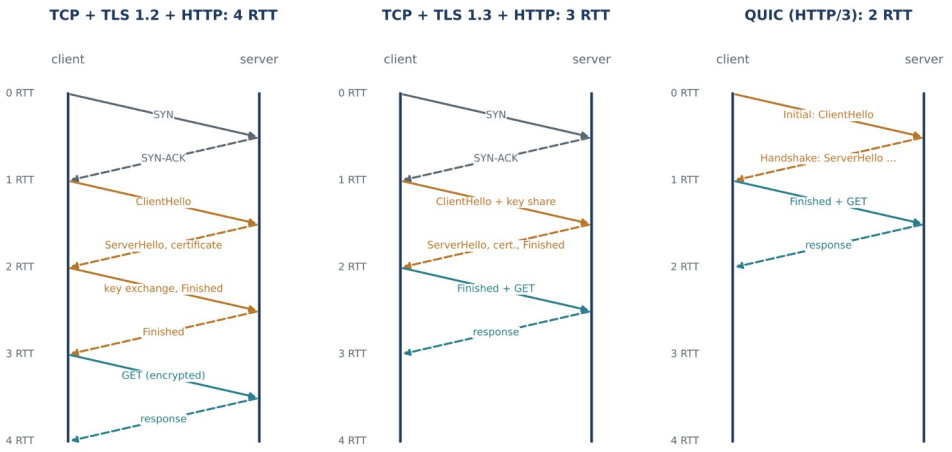


Figura 1. Drumuri dus-întors înainte de primul octet al răspunsului pentru o conexiune nouă. TLS 1.2 adaugă două drumuri dus-întors la cel al TCP, TLS 1.3 adaugă unul, iar QUIC contopește handshake-urile de transport și de securitate.

5.2 Certificatele și lanțul de încredere

Criptarea singură nu este suficientă: un om-la-mijloc (man in the middle) ar putea face câte un handshake cu fiecare parte și ar putea retransmite datele. De aceea serverul prezintă un **certificat X.509** care leagă numele său (de exemplu *www.example.org*) de cheia sa publică, semnat de o **autoritate de certificare** (CA). Browserul verifică faptul că numele se potrivește, că datele calendaristice sunt valide și că lanțul de semnături duce — prin unul sau mai multe certificate intermediare — la un **certificat rădăcină** din depozitul de încredere livrat cu sistemul de operare sau cu browserul. Certificatele sunt astăzi ieftine și automatizate (Let's Encrypt emite certificate gratuite prin protocolul ACME), iar fiecare certificat emis este înregistrat public (**Certificate Transparency**), astfel încât o CA nu poate emite în secret unul pentru numele dumneavoastră.

5.3 Ce ascunde HTTPS și ce nu ascunde

Vizibil pentru un observator de pe traseu	Ascuns de TLS
adresele IP și porturile (443) numele serverului, în câmpul **SNI** (Server Name Indication) al ClientHello — și în interogarea DNS care l-a precedat dimensiunile și momentele înregistrărilor, numărul de conexiuni	metoda, calea URL și interogarea (query) toate antetele, inclusiv cookie-urile corpurile cererilor și ale răspunsurilor

Două rafinamente închid golurile rămase. **HSTS** (antetul `Strict-Transport-Security`) îi spune browserului să folosească doar HTTPS pentru un site, astfel încât o primă vizită tastată ca http:// să nu poată fi interceptată. **Encrypted Client Hello** (ECH), aflat acum în curs de implementare, ascunde și numele serverului. Rețineți, în fine, că TLS protejează traseul, nu capetele: un server poate totuși folosi abuziv ceea ce primește.

6. Autentificare la distanță în clar: TELNET

Folosirea unui calculator de la distanță a fost una dintre primele aplicații de rețea. **TELNET** (RFC 854, 1983; portul 23) oferă unui utilizator de pe un host o sesiune interactivă de terminal pe alt host, peste TCP. Definește un **terminal virtual de rețea** (NVT): un terminal standard, orientat pe linii, cu

ASCII pe 7 biți și terminatori de linie CR LF, astfel încât niciuna dintre părți să nu trebuiască să cunoască terminalul real al celeilalte.

Funcționalitățile care depășesc NVT sunt convenite prin **negocierea opțiunilor**, transportată în fluxul de date și introdusă prin octetul **IAC** (*interpret as command* — interpretează ca o comandă, 255). Oricare parte poate oferi sau cere o opțiune printr-o comandă de trei octeți; cealaltă parte acceptă sau refuză.

Octeți	Semnificație
`FF FB 01`	IAC **WILL** ECHO — „Eu (serverul) voi face ecoul a ceea ce tastați” (clientul nu mai face ecou local, ca în cazul unei parole)
`FF FD 01`	IAC **DO** ECHO — „te rog, fă ecou”
`FF FC 01`, `FF FE 01`	IAC **WONT** / **DONT** ECHO — refuz sau oprire
`FF FD 03`	IAC DO SUPPRESS-GO-AHEAD (opțiunea 3): funcționare full-duplex
`FF FA 18 … FF F0`	IAC SB TERMINAL-TYPE … IAC SE: o sub-negociere care transportă numele terminalului

Slăbiciunea fatală a TELNET este că **totul circulă în clar**, inclusiv numele de utilizator și parola: oricine poate captura pachete pe traseu — într-o rețea Wi-Fi partajată, într-un router compromis — le citește și poate chiar injecta comenzi în sesiune. Veți vedea asta cu propriii ochi în laborator. TELNET supraviețuiește astăzi doar pe echipamente de rețea vechi și pe dispozitive încorporate (embedded), iar această supraviețuire are un preț: în 2016 botnetul **Mirai** a preluat controlul a sute de mii de camere și routere pur și simplu încercând parolele din fabrică pe porturile lor TELNET. Programul `telnet` rămâne util într-un alt rol — ca instrument pentru tastarea manuală a protocoalelor text, așa cum ați făcut cu `nc` în Laboratorul 1.

7. SSH: shell securizat, tuneluri securizate

SSH (Secure Shell; Tatu Ylönen, 1995; SSH-2 standardizat în RFC-urile 4251–4254, 2006; portul 22) a înlocuit TELNET și *comenzile r* din BSD (rlogin, rsh, rcp). Oferă autentificare la distanță, execuție de comenzi la distanță și transfer de fișiere, toate peste o singură conexiune TCP criptată și autentificată. OpenSSH, dezvoltat de proiectul OpenBSD, este implementarea prezentă pe aproape toate sistemele Linux și macOS și, din 2018, o componentă integrată a Windows.

7.1 Trei niveluri

Nivel (RFC)	Rol
Protocolul nivelului de transport (RFC 4253)	schimbul de versiuni (`SSH-2.0-OpenSSH_9.6…`), negocierea algoritmilor, schimbul de chei (Diffie–Hellman pe curbe eliptice; versiunile recente OpenSSH adaugă o componentă post-cuantică), **autentificarea serverului** prin cheia sa de host, apoi criptarea și integritatea a tot ce urmează
Protocolul de autentificare a utilizatorului (RFC 4252)	clientul dovedește cine este utilizatorul: **parolă**, **cheie publică** sau keyboard-interactive (de exemplu un cod de unică folosință)
Protocolul de conexiune (RFC 4254)	multiplexează multe **canale** pe o singură conexiune: un shell interactiv, o comandă la distanță, subsistemul SFTP, conexiuni TCP redirecționate

7.2 Cheile de host: este acesta chiar serverul?

Fiecare server SSH are o **pereche de chei de host** de lungă durată. În timpul schimbului de chei, serverul semnează schimbul cu cheia sa privată de host; clientul verifică semnătura cu cheia publică de host. Dar de unde știe clientul că această cheie publică aparține serverului real? De obicei nu există o autoritate de certificare. La prima conexiune, OpenSSH afișează **amprenta (fingerprint)** cheii (de exemplu `SHA256:3j8f…`) și îi cere utilizatorului să o confirme; cheia este apoi stocată în `~/.ssh/known_hosts`, iar fiecare conexiune ulterioară trebuie să prezinte aceeași cheie. Acest model se numește **încredere la prima utilizare** (trust on first use, TOFU). Dacă cheia se schimbă vreodată, clientul refuză să se conecteze, cu avertismentul „REMOTE HOST IDENTIFICATION HAS CHANGED!” — fie serverul a fost reinstalat, fie cineva se află la mijloc. Un administrator poate publica amprentele în avans sau în DNS (înregistrări SSHFP).

7.3 Autentificarea utilizatorului cu cheie publică

Parolele pot fi ghicite, furate prin phishing sau capturate de un server compromis. Cu **autentificarea cu cheie publică**, utilizatorul generează o pereche de chei (`ssh-keygen -t ed25519`), instalează cheia **publică** pe server în `~/.ssh/authorized_keys` și păstrează cheia **privată** pe laptop, protejată printr-o frază de acces (passphrase). Pentru autentificare, clientul semnează cu cheia privată o valoare care depinde de sesiunea curentă; serverul verifică semnătura cu cheia publică stocată. Cheia privată nu părăsește niciodată laptopul, iar semnătura este inutilă în orice altă sesiune. Autentificarea cu cheie publică este norma pentru administratori și pentru instrumentele automate precum Git.

7.4 Canale: shell-uri, comenzi, fișiere și tuneluri

Comandă	Ce face
`ssh user@host`	shell interactiv pe host-ul de la distanță
`ssh user@host 'df -h'`	rulează o comandă la distanță și afișează local rezultatul
`scp file user@host:dir/`, `sftp user@host`	copiază fișiere; **SFTP** (SSH File Transfer Protocol) este un subsistem al SSH — în ciuda numelui, nu are nicio legătură cu FTP
`ssh -L 8080:intranet:80 user@gw`	**redirecționare locală**: conexiunile către localhost:8080 sunt transportate prin conexiunea SSH și deschise de gateway către intranet:80
`ssh -R 9000:localhost:3000 user@server`	**redirecționare la distanță**: portul 9000 al serverului duce înapoi la un program de pe mașina dumneavoastră
`ssh -D 1080 user@gw`	**redirecționare dinamică**: un proxy SOCKS pe portul 1080; gateway-ul deschide orice conexiuni cer aplicațiile dumneavoastră

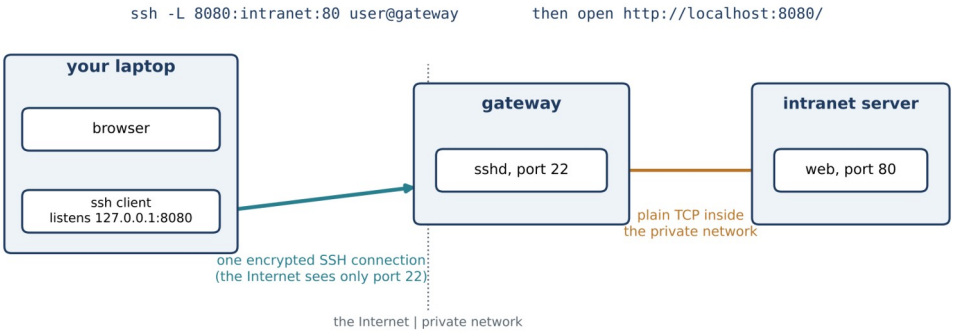


Figura 2. Redirecționarea locală a porturilor. Browserul vorbește cu un port de pe laptop; conexiunea SSH transportă octeții, criptați, până la gateway, care deschide o conexiune TCP obișnuită în interiorul rețelei private.

Redirecționarea ilustrează într-un mod nou stratificarea din Cursul 2: un canal SSH este un flux de octeți fiabil transportat în interiorul unei conexiuni TCP, iar fluxul TCP al unei aplicații poate fi transportat în interiorul lui. Observați ce protejează tunelul — doar porțiunea dintre clientul SSH și serverul SSH. Ultima porțiune, de la gateway la serverul din intranet, este o conexiune obișnuită.

8. FTP: două conexiuni și un dialog cu stare

FTP (File Transfer Protocol; RFC 959, 1985, de J. Postel și J. Reynolds, pe baza unor lucrări începute în 1971) este una dintre cele mai vechi aplicații Internet încă în uz. Un utilizator se autentifică pe un host de la distanță și listează, descarcă, încarcă, redenumește și șterge fișiere. Trei trăsături îl deosebesc de HTTP: folosește **două tipuri de conexiuni TCP**, este **cu stare (stateful)** și spune precis cum sunt **organizate** datele transferate.

8.1 Gestionarea conexiunilor: control și date

Clientul deschide o **conexiune de control** către portul **21** al serverului și o păstrează pe toată durata sesiunii. Comenzile și răspunsurile circulă pe ea ca linii de text NVT-ASCII, exact ca în TELNET. Pentru fiecare fișier și fiecare listare de director se deschide o **conexiune de date** separată, folosită pentru acel singur transfer și apoi închisă. Prin urmare, FTP își trimite informațiile de control **în afara benzii (out of band)** — pe o conexiune separată de date — în timp ce HTTP trimite antetele și corpul **în bandă (in band)**, pe o singură conexiune.

- **Modul activ** (cel original): clientul ascultă pe un port și îl anunță cu PORT `h1,h2,h3,h4,p1,p2` — adresa IP a clientului ca patru octeți zecimali și portul ca doi, port = p1 × 256 + p2. Serverul **se conectează apoi la client**, de pe portul său 20.
- **Modul pasiv**: clientul trimite PASV; serverul deschide un port de ascultare și îl anunță în răspunsul `227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)`; **clientul se conectează la server**. Ambele conexiuni sunt deschise de client.
- **Versiunile extinse** pentru IPv6 (RFC 2428): `EPRT |2|2001:db8::5|51001|` și EPSV, cu răspunsul `229 Entering Extended Passive Mode (|||50123|)`.

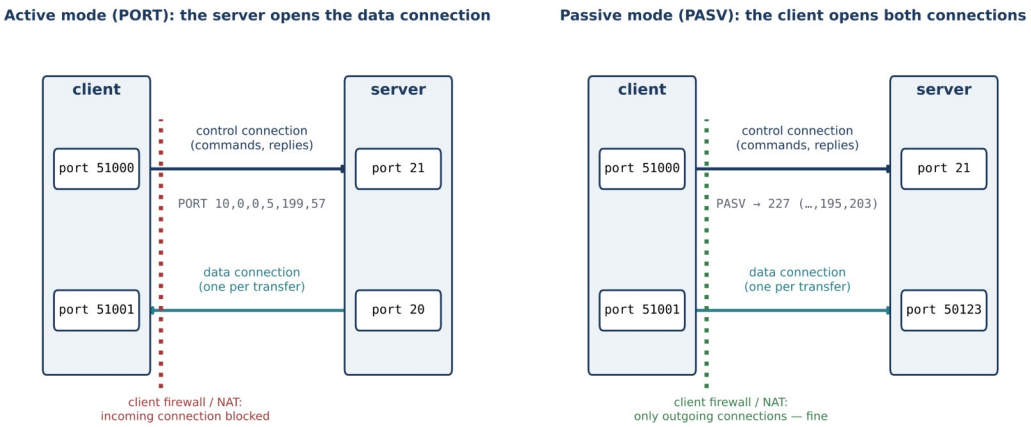


Figura 3. FTP activ și pasiv. În modul activ, serverul trebuie să deschidă o conexiune către client, pe care un firewall sau un router NAT aflat în fața clientului o blochează în mod normal; în modul pasiv, ambele conexiuni sunt de ieșire pentru client.

De ce o conexiune de date nouă pentru fiecare fișier? În modul de transfer implicit, sfârșitul fișierului este marcat prin **închiderea conexiunii de date** — cel mai simplu răspuns posibil la problema încadrării din Cursul 3. Aceasta explică și dificultatea FTP cu firewall-urile și NAT: conexiunea de date este deschisă către o adresă și un port scrise *în interiorul* unui mesaj de control. Un client aflat în spatele unui router de acasă își anunță adresa privată în PORT, iar încercarea de conectare a serverului nu îl poate atinge. Modul pasiv rezolvă partea clientului; un server aflat în spatele NAT trebuie să-și anunțe adresa publică și un interval fix de porturi pasive; iar multe routere conțin un **gateway la nivel de aplicație** (ALG) pentru FTP, care citește și rescrie comenzile PORT — o încălcare a stratificării la care vom reveni săptămâna viitoare, odată cu NAT.

8.2 Comenzi și răspunsuri

Comenzile au cel mult patru litere, urmate de argumente și CR LF. Fiecare răspuns începe cu un cod de trei cifre, a cărui primă cifră spune cum au decurs lucrurile, iar a doua cifră spune la ce se referă răspunsul; un răspuns se poate întinde pe mai multe linii, cu `123-` pe prima și `123 ` pe ultima.

Grup	Comenzi
Acces	`USER`, `PASS`, `ACCT`, `QUIT`; *FTP anonim*: utilizatorul `anonymous`, o adresă de e-mail ca parolă
Navigare	`PWD`, `CWD dir`, `CDUP`, `LIST`, `NLST`, `MLSD` (listare citibilă de mașină, RFC 3659)
Transfer	`RETR` (descărcare), `STOR` (încărcare), `APPE` (adăugare la final), `REST n` (repornire de la octetul n), `ABOR`
Gestionarea fișierelor	`DELE`, `RNFR` + `RNTO` (redenumire), `MKD`, `RMD`, `SIZE`, `MDTM`
Parametri	`TYPE`, `STRU`, `MODE` (secțiunea 8.3), `PORT`, `PASV`, `EPRT`, `EPSV`
Informații	`SYST`, `FEAT` (extensii suportate, RFC 2389), `HELP`, `NOOP`

Prima cifră	Semnificație	Exemple
1	pozitiv preliminar: a început, așteptați un alt răspuns	150 Opening data connection
2	finalizare pozitivă	200 OK · 220 Service ready · 226 Transfer complete · 230 Logged in · 227 Entering Passive Mode

Prima cifră	Semnificație	Exemple
3	pozitiv intermediar: trimiteți mai mult	331 User name okay, need password · 350 Requested action pending (after RNFR)
4	negativ tranzitoriu: încercați din nou mai târziu	421 Service not available · 425 Can't open data connection · 450 File busy
5	negativ permanent: nu repetați la fel	500 Syntax error · 530 Not logged in · 550 File unavailable

A doua cifră grupează răspunsurile după subiect: x0 sintaxă, x1 informații, x2 conexiuni, x3 autentificare, x5 sistemul de fișiere. Un program client poate astfel reacționa la orice răspuns doar pe baza primei cifre — aceeași idee ca la clasele de stare din HTTP, care a împrumutat-o de la FTP și SMTP.

8.3 Organizarea datelor: tipuri, structuri și moduri

Calculatoarele din 1985 nu se înțelegeau aproape în nicio privință — dimensiunea octetului, setul de caractere, terminatorii de linie, structura înregistrărilor. De aceea RFC 959 le permite celor două părți să convină, înainte de un transfer, asupra a trei proprietăți ale datelor.

Comandă	Valori	Semnificație
TYPE — reprezentarea datelor	**A** ASCII (implicit) · **E** EBCDIC · **I** imagine · **L n** dimensiune locală a octetului	În tipul A, fișierul este trimis ca NVT-ASCII cu terminatori de linie **CR LF**, convertiți din și în convenția proprie a fiecărui host. Tipul **I** trimite octeții nemodificați — singura alegere corectă pentru orice nu este text simplu (imagini, arhive, programe). Un control de format (non-print, TELNET, controlul de car ASA) poate însoți tipurile A și E.
STRU — structura fișierului	**F** fișier (implicit) · **R** înregistrare · **P** pagină	Un fișier este o secvență continuă de octeți; sau o secvență de înregistrări (pentru sistemele orientate pe înregistrări); sau de pagini indexate independente.
MODE — modul de transmisie	**S** flux (implicit) · **B** bloc · **C** comprimat	Flux (stream): octeți bruți, sfârșitul fișierului = închiderea conexiunii de date. Bloc: un antet (descriptor, număr de octeți) înaintea fiecărui bloc, ceea ce permite **marcaje de repornire** și păstrarea deschisă a conexiunii de date. Comprimat: codificarea secvențelor (run-length) de octeți repetați.

Astăzi doar TYPE A față de TYPE I mai contează în practică, iar diferența contează mult: transferul unei fotografii în mod ASCII între Windows și Linux transformă fiecare octet 0x0A în 0x0D 0x0A sau elimină fiecare 0x0D și corupe fișierul. Clienții moderni comută automat pe binar.

8.4 Stare, securitate și FTP astăzi

Un server FTP păstrează o **stare** per sesiune: dacă utilizatorul s-a autentificat, directorul curent, valorile TYPE, MODE și STRU în vigoare, un RNFR în așteptare. HTTP nu păstrează niciuna; acesta este unul dintre motivele pentru care HTTP scalează mai ușor la milioane de clienți.

Ca și TELNET, FTP-ul simplu trimite **parola în clar** pe conexiunea de control. Există trei alternative sigure: **FTPS** — FTP peste TLS, fie actualizat cu comanda AUTH TLS (RFC 4217), fie pe propriul port

990; **SFTP** și **scp** peste SSH; și, pentru majoritatea descărcărilor, pur și simplu **HTTPS**. Browserele majore au eliminat suportul pentru FTP în 2021. FTP supraviețuiește în arhivele științifice și publice (adesea anonime), în găzduirea web și în sistemele industriale și moștenite (legacy) — motiv pentru care fiecare inginer de rețea trebuie să poată citi în continuare un dialog FTP.

Rezumat

- HTTP rămâne fără stare; **cookie-urile** leagă un browser de starea păstrată de server. Cookie-urile terțe permit urmărirea între site-uri.
- **Cache-urile** — browser, proxy, CDN — reduc timpul de răspuns și încărcarea legăturii. **Prospețimea** (`Cache-Control: max-age`) și **validarea** (GET condiționat cu `ETag` / `If-None-Match`, cu răspunsul **304**) mențin copiile corecte.
- **HTTP/2** multiplexează cadre binare ale mai multor fluxuri pe o singură conexiune TCP și comprimă antetele; blocarea la capul cozii la nivelul TCP rămâne. **HTTP/3** rulează peste **QUIC**: fluxuri independente peste UDP, TLS 1.3 integrat, stabilire a conexiunii în 1-RTT și 0-RTT, migrarea conexiunii.
- **HTTPS** = HTTP peste **TLS**: confidențialitate, integritate, autentificarea serverului prin **certificate** și un lanț de încredere. TLS 1.3 are nevoie de un drum dus-întors; numele serverului (SNI), adresele și dimensiunile rămân vizibile.
- **TELNET** oferă un terminal virtual de rețea și negociază opțiuni prin secvențe IAC, dar trimite totul — inclusiv parolele — în clar.
- **SSH** are trei niveluri: transport (schimbul de chei, autentificarea serverului prin cheia de host, criptare), autentificarea utilizatorului (parolă sau **cheie publică**) și conexiune (canale). Cheile de host sunt de încredere la prima utilizare. Canalele transportă shell-uri, comenzi, **scp/SFTP** și **redirecționarea porturilor**.
- **FTP** folosește o **conexiune de control** persistentă (portul 21) și câte o **conexiune de date** pentru fiecare transfer, deschisă de server (**activ**, PORT) sau de client (**pasiv**, PASV). Răspunsurile au coduri de trei cifre. TYPE, STRU și MODE definesc modul în care datele sunt reprezentate, structurate și transmise. FTP este cu stare și trimite parolele în clar; FTPS și SFTP sunt alternativele sigure.

Termeni-cheie

fără stare (stateless) · cookie · Set-Cookie · cookie de sesiune · HttpOnly · Secure · SameSite · cookie primar / terț · cache web · proxy · proxy invers · rată de hit · prospețime · max-age · Age · validare · GET condiționat · ETag · If-None-Match · If-Modified-Since · 304 Not Modified · CDN · HTTP/2 · cadru (frame) · flux (stream) · multiplexare · HPACK · blocare la capul cozii · HTTP/3 · QUIC · QPACK · 0-RTT · ID de conexiune · Alt-Svc · HTTPS · TLS · ClientHello · ServerHello · key share · Finished · certificat · autoritate de certificare · depozit de certificate rădăcină · lanț de încredere · Certificate Transparency · SNI · HSTS · ECH · TELNET · terminal virtual de rețea · IAC · WILL / WONT / DO / DONT · SSH · cheie de host · amprentă (fingerprint) · known_hosts · încredere la prima utilizare · autentificare cu cheie publică · authorized_keys · canal · scp · SFTP · redirecționare locală / la distanță / dinamică · FTP · conexiune de control · conexiune de date · mod activ · mod pasiv · PORT · PASV · EPRT · EPSV · cod de răspuns · TYPE A / TYPE I · STRU · MODE · mod flux / bloc / comprimat · FTP anonim · FTPS · gateway la nivel de aplicație

Întrebări de recapitulare

1. HTTP este fără stare, totuși un magazin vă ține minte coșul. Unde anume este stocat coșul și ce circulă în fiecare cerere?
2. De ce protejează `HttpOnly` un cookie de sesiune împotriva unui script injectat în pagină? Împotriva a ce protejează `Secure`?
3. Un cache deține o copie al cărei `max-age` a expirat. Trebuie să descarce din nou obiectul? Explicați alternativa.
4. Ce antet folosește un server pentru a-i spune unui cache că o pagină poate fi stocată, dar trebuie verificată înainte de fiecare utilizare?
5. HTTP/2 multiplexează fluxuri pe o singură conexiune. De ce poate un singur pachet pierdut să le întârzie totuși pe toate și cum evită HTTP/3 acest lucru?
6. Enumerați ce află un observator de pe traseu despre o vizită HTTPS și ce rămâne ascuns.
7. Ce ar merge prost dacă TLS ar cripta datele, dar serverul nu ar prezenta niciun certificat?
8. Ce afișează SSH la prima conexiune către un server nou și ce înseamnă avertismentul „REMOTE HOST IDENTIFICATION HAS CHANGED”?
9. La autentificarea cu cheie publică, ce stochează serverul, ce păstrează clientul și ce se trimite în timpul autentificării?
10. De ce eșuează FTP-ul în mod activ pentru un client aflat în spatele unui router NAT de acasă, în timp ce modul pasiv funcționează?
11. De ce trebuie transferată o imagine PNG cu `TYPE I` și nu cu `TYPE A`?
12. În modul flux al FTP, de unde știe receptorul unde se termină un fișier? Comparați cu HTTP/1.1.

Exerciții

1. Legătura de acces a unei instituții are 50 Mbit/s; utilizatorii cer 45 de obiecte pe secundă, de 1 Mbit în medie; întârzierea Internet (drum dus-întors plus server) este 0,8 s. (a) Calculați intensitatea traficului și întârzierea medie pe legătura de acces cu formula d = (L/R) / (1 − ρ). (b) Se instalează un cache cu o rată de hit de 50 %; întârzierea în LAN pentru un hit este de 5 ms. Calculați noul timp mediu de răspuns.
2. RTT = 60 ms. Calculați timpul până la primul octet al răspunsului pentru o conexiune nouă cu (a) HTTP/1.1 peste TCP fără TLS; (b) TCP + TLS 1.2; (c) TCP + TLS 1.3; (d) QUIC; (e) QUIC cu reluare 0-RTT. Ignorați timpii de transmisie și de server.
3. Decodificați octeții `FF FB 01 FF FB 03 FF FD 18` primiți la începutul unei sesiuni TELNET.
4. Un client trimite `PORT 192,168,1,20,78,52`. La ce adresă și port se va conecta serverul? De ce va eșua acest lucru dacă clientul se află în spatele NAT, iar serverul în Internet?
5. Un utilizator descarcă o listare de director și apoi 10 fișiere de pe un server FTP în mod pasiv. Câte conexiuni TCP se deschid în total? Câte handshake-uri în trei pași?

Răspunsuri

1. (a) ρ = 45 / 50 = 0,9; L/R = 1 Mbit / 50 Mbit/s = 20 ms; d = 20 / 0,1 = **200 ms**; răspuns ≈ 0,8 + 0,2 = 1,0 s. (b) Doar 22,5 obiecte/s traversează legătura: ρ = 0,45, d = 20 / 0,55 ≈ 36 ms. Media = 0,5 × 0,005 + 0,5 × (0,8 + 0,036) ≈ **0,42 s**.

2. (a) 2 RTT = 120 ms (handshake, apoi cerere–răspuns). (b) 4 RTT = 240 ms. (c) 3 RTT = 180 ms. (d) 2 RTT = 120 ms. (e) 1 RTT = 60 ms: cererea pleacă odată cu primul pachet.
3. IAC WILL ECHO (serverul va face ecou), IAC WILL SUPPRESS-GO-AHEAD (full duplex), IAC DO TERMINAL-TYPE (opțiunea 24 = 0x18: „spune-mi tipul terminalului tău”).
4. Către 192.168.1.20, portul 78 × 256 + 52 = **20 020**. 192.168.1.20 este o adresă privată, fără sens în Internet; nici chiar adresa publică a routerului nu ar ajuta, deoarece routerul nu are nicio intrare de translatare pentru o conexiune *de intrare* către portul 20 020.
5. O conexiune de control, o conexiune de date pentru listare și câte una pentru fiecare fișier: **12 conexiuni**, deci 12 handshake-uri.

Lecturi și pașii următori

Obligatoriu

- Kurose și Ross, *Computer Networking: A Top-Down Approach*, ediția a 8-a, Capitolul 2: interacțiunea utilizator–server cu cookie-uri, cache-ul web, HTTP/2 și HTTP/3; Capitolul 8: securizarea conexiunilor TCP cu TLS.

Recomandat

- RFC 6265 (cookie-uri); RFC 9111 (cache-ul HTTP); RFC 9113 (HTTP/2); RFC 9000 și RFC 9114 (QUIC, HTTP/3); RFC 8446 (TLS 1.3), secțiunile 1–2.
- RFC 854 (TELNET); RFC 4251 (arhitectura SSH); RFC 959 (FTP), secțiunile 3 și 4 — citiți 3.1 despre reprezentarea datelor și 3.4 despre modurile de transmisie.
- D. J. Barrett, R. E. Silverman, R. G. Byrnes, *SSH, The Secure Shell: The Definitive Guide*, ediția a 2-a, capitolele 1–3.

Laboratorul 4 (săptămâna aceasta)

Urmăriți un cookie de sesiune și un GET condiționat, inspectați un handshake TLS și un certificat, capturați în clar o parolă de tip TELNET, vă autentificați cu chei SSH și construiți un tunel și urmăriți cele două conexiuni ale FTP — mai întâi cu instrumente standard, apoi cu un client FTP propriu, scris în C.

Cursul următor

Nivelul aplicație III: poșta electronică (SMTP, formatul mesajelor și MIME, POP3 și IMAP), știrile Usenet, sistemul de nume de domenii (DNS) și translatarea adreselor de rețea (NAT).

PARTEA B · SĂPTĂMÂNA 4

Seminarul 4: cache-uri, handshake-uri, tuneluri și două conexiuni

Sesiune de probleme de două ore (2 × 50 de minute) care însoțește Cursul 4

Seminarul transformă Cursul 4 în calcule, citirea atentă a octeților de protocol și mici proiecte. Lucrați în perechi; câte o pereche prezintă fiecare problemă.

La sfârșitul seminarului ar trebui să puteți

- urmări cookie-urile schimbate într-o scurtă sesiune de navigare;
- calcula efectul unui cache și urmări în timp prospețimea unei copii din cache;
- compara HTTP/1.1 și HTTP/2 atunci când un obiect mare împarte o conexiune cu obiecte mici;
- număra drumurile dus-întors ale handshake-urilor TLS și spune ce vede un observator dintr-o vizită HTTPS;
- decodifica octeții de opțiuni TELNET și raționa despre cheile de host și tunelurile SSH;
- citi un dialog FTP, decodifica PORT și PASV și explica problemele legate de tipul de transfer și de NAT.

Plan

Pas	Activitate	Timp
A	Problema 1: urmărirea cookie-urilor	8 min
B	Problemele 2–3: cache-uri și prospețime	17 min
C	Problema 4: o conexiune, multe fluxuri	10 min
D	Problemele 5–6: TLS — drumuri dus-întors, observatori și certificate	15 min
	Pauză	10 min
E	Problemele 7–9: octeți TELNET, chei de host și tuneluri SSH	20 min
F	Problemele 10–12: citirea FTP, tipuri de transfer, multe fișiere mici	20 min
	Temă	(în F)

A. Urmărirea cookie-urilor

Problema 1

Un student cu un browser gol vizitează `https://shop.example.ro/`, apoi `https://shop.example.ro/cart`, apoi un site de știri `https://news.example.com/` a cărui pagină include o imagine de la `https://ads.tracker.net/pixel.gif`. Pagina magazinului include același pixel. Răspunsurile conțin:

```
shop.example.ro    Set-Cookie: sid=a91f; Path=/; Secure; HttpOnly; SameSite=Lax
ads.tracker.net    Set-Cookie: uid=77c2; Max-Age=31536000; Secure; SameSite=None
```

1. Ce antet `Cookie:` conține fiecare dintre cereri (cele patru cereri de pagină și de pixel pe magazin, cele două de pe site-ul de știri)?

2. Ce cookie dispare la închiderea browserului? Care durează un an?
3. Ce poate afla ads.tracker.net și din ce antete de cerere?
4. Ce se schimbă dacă browserul blochează cookie-urile terțe?

> **Soluție**
> 1. Prima pagină a magazinului: niciun cookie. Pixelul din pagina magazinului: niciunul (prima vizită la tracker). `/cart`: `Cookie: sid=a91f`; pixelul său: `Cookie: uid=77c2`. Pagina de știri: niciunul (niciun cookie pentru news.example.com). Pixelul din pagina de știri: `Cookie: uid=77c2`.
> 2. `sid` este un cookie de sesiune (fără Expires sau Max-Age); `uid` durează 31 536 000 s = un an.
> 3. Tracker-ul primește `uid=77c2` de la ambele site-uri, împreună cu antetul `Referer` care numește pagina ce a încorporat pixelul (sau cel puțin site-ul ei): află că același browser a vizitat coșul magazinului și site-ul de știri și poate construi în timp un profil.
> 4. Browserul nici nu stochează, nici nu trimite `uid` într-un context terț; fiecare site vede cererea pentru pixel fără un identificator stabil, iar urmărirea între site-uri prin cookie încetează. (Tracker-ele încearcă atunci alte tehnici, cum ar fi amprentarea browserului — fingerprinting.)

B. Cache-uri și prospețime

Problema 2

Rețeaua unei facultăți ajunge la Internet printr-o legătură de **200 Mbit/s**. Utilizatorii cer **160 de obiecte pe secundă** de **1 Mbit** în medie; serverele de origine se află în medie la **0,6 s** distanță (drum dus-întors plus timpul serverului). Folosiți d = (L/R) / (1 − ρ) pentru legătura de acces și ignorați întârzierile din interiorul facultății.

1. Calculați intensitatea traficului pe legătura de acces și timpul mediu de răspuns.
2. Opțiunea 1: modernizarea legăturii la 1 Gbit/s. Calculați noul timp de răspuns.
3. Opțiunea 2: un proxy cache cu o rată de hit de 35 %; un hit primește răspuns în 4 ms. Calculați noul timp de răspuns.
4. Ce opțiune ați recomanda și ce altceva mai economisește cache-ul?

> **Soluție**
> 1. ρ = 160 × 1 / 200 = 0,8; L/R = 5 ms; d = 5 / 0,2 = 25 ms. Răspuns ≈ 0,6 + 0,025 = **0,625 s**.
> 2. ρ = 0,16; L/R = 1 ms; d = 1 / 0,84 ≈ 1,2 ms. Răspuns ≈ **0,601 s** — aproape niciun câștig: domină cele 0,6 s ale traseului prin Internet.
> 3. 104 obiecte/s traversează legătura: ρ = 0,52, d = 5 / 0,48 ≈ 10,4 ms. Răspuns = 0,35 × 0,004 + 0,65 × (0,6 + 0,0104) ≈ **0,40 s**.
> 4. Cache-ul: este mai ieftin și reduce timpul de răspuns cu o treime, deoarece un hit evită cu totul serverul îndepărtat, lucru pe care nicio legătură de acces mai rapidă nu îl poate face. Reduce de asemenea încărcarea legăturii (de la 80 % la 52 %) și a serverelor de origine. Limitele sale: paginile dinamice și personale nu pot fi păstrate în cache, iar HTTPS ascunde conținutul de un proxy instituțional, cu excepția cazului în care browserul este configurat să aibă încredere în acesta.

Problema 3

Cache-ul unui browser preia `/timetable.html` la 10:00:00. Răspunsul conține `Cache-Control: max-age=600` și `ETag: "v1"`; corpul are 40 KB. Pagina se schimbă pe server la 10:25:00, primind `ETag: "v2"`. Utilizatorul deschide pagina la 10:05, 10:12, 10:20 și 10:30. Pentru fiecare vizită, spuneți dacă browserul contactează serverul, ce trimite, ce primește și câți octeți de corp traversează rețeaua.

> **Soluție**
> **10:05** — copie proaspătă (vârsta 300 s < 600 s): servită din cache, nicio cerere, 0 octeți.
> **10:12** — expirată (vârsta 720 s): GET condiționat cu `If-None-Match: "v1"`; nemodificată, deci **304 Not Modified**, 0 octeți de corp; copia este din nou proaspătă până la 10:22.
> **10:20** — proaspătă (revalidată la 10:12): servită din cache, nicio cerere.
> **10:30** — expirată: GET condiționat cu `If-None-Match: "v1"`; pagina s-a schimbat, deci **200 OK** cu `ETag: "v2"` și corpul de 40 KB. Total: 40 KB transferați o singură dată în loc de patru ori, cu prețul a două cereri condiționate mici. Observați riscul: între 10:25 și 10:30 utilizatorul ar fi putut vedea pagina veche — `max-age` schimbă prospețime pe viteză.

C. O conexiune, multe fluxuri

Problema 4

O pagină are nevoie de un clip video de **1 MB** și de **10 pictograme mici de câte 10 KB**, toate de la același server, printr-o strangulare (bottleneck) de **10 Mbit/s**. Browserul cere mai întâi clipul video, apoi pictogramele. Ignorați drumurile dus-întors și antetele; 1 MB = 8 Mbit, 10 KB = 80 kbit.

1. HTTP/1.1 pe o singură conexiune persistentă, fără trucuri de pipelining: când sosește prima pictogramă? Dar ultima?
2. HTTP/2 pe o singură conexiune, serverul întrețesând cadrele tuturor celor 11 fluxuri în părți egale: când sosesc pictogramele? Când se termină clipul video?
3. În HTTP/2, un singur segment TCP al clipului video se pierde, iar retransmisia lui durează 100 ms. Ce fluxuri sunt întârziate? Ce s-ar întâmpla cu HTTP/3?

> **Soluție**
> 1. Clipul video durează 8 / 10 = 0,8 s; apoi fiecare pictogramă 8 ms. Prima pictogramă la **0,808 s**, ultima la **0,88 s**: blocare la capul cozii.
> 2. Fiecare dintre cele 11 fluxuri primește 10/11 Mbit/s ≈ 0,91 Mbit/s; o pictogramă (80 kbit) se termină după aproximativ **88 ms**, toate zece împreună. Clipul video are apoi întreaga legătură și se termină la **0,88 s**, ca înainte — totalul este același, dar obiectele mici nu mai așteaptă.
> 3. TCP livrează octeții în ordine, astfel încât fiecare flux ale cărui date urmează segmentului pierdut în fluxul de octeți așteaptă aproximativ 100 ms — inclusiv pictogramele. Cu HTTP/3 peste QUIC, fiecare flux este livrat independent: doar fluxul video așteaptă.

D. TLS: drumuri dus-întors, observatori și certificate

Problema 5

Un student aflat pe rețeaua Wi-Fi a unei cafenele deschide `https://mail.example.com/inbox?folder=work`. RTT până la server = 50 ms; răspunsul DNS este deja în cache.

1. Cât durează până la primul octet al răspunsului cu TCP + TLS 1.3? Cu TLS 1.2? Cu HTTP/3?
2. Enumerați ce poate vedea punctul de acces al cafenelei.
3. A doua zi dimineață, browserul reia sesiunea TLS cu date 0-RTT. De ce acceptă serverele ca date 0-RTT doar cereri sigure, idempotente?

> **Soluție**
> 1. TLS 1.3: handshake TCP (1 RTT) + handshake TLS (1 RTT) + cerere–răspuns (1 RTT) = **150 ms**. TLS 1.2: **200 ms**. HTTP/3: **100 ms**.
> 2. Adresele IP ale studentului și ale serverului, porturile (443), numele `mail.example.com` din SNI-ul ClientHello (și din interogarea DNS anterioară, dacă aceasta nu a fost criptată), momentul, numărul și dimensiunile înregistrărilor criptate. Nu sunt vizibile: calea `/inbox`, interogarea `folder=work`, cookie-urile, conținutul cutiei poștale.
> 3. Datele 0-RTT circulă înainte ca handshake-ul să fi dovedit că clientul este activ, astfel încât un atacator care le-a înregistrat le poate **reda (replay)**. Un GET redat nu face niciun rău; un „trimite acest e-mail” sau o plată redată ar face.

Problema 6

Serverul www.shop.ro trimite un certificat pentru `www.shop.ro`, emis de o CA intermediară *R11*, al cărei certificat este emis de rădăcina *ISRG Root X1*, prezentă în depozitul de încredere al browserului. Pentru fiecare caz, acceptă browserul conexiunea? De ce?

1. Totul ca în descriere, toate certificatele în perioada lor de valabilitate.
2. Utilizatorul a tastat `shop.ro` (fără www), iar certificatul conține doar `www.shop.ro`.
3. Serverul a uitat să trimită certificatul intermediar R11.
4. Certificatul final (leaf) a expirat ieri.
5. Un atacator de pe rețeaua Wi-Fi a cafenelei prezintă un certificat pentru www.shop.ro pe care l-a semnat cu propria cheie.

> **Soluție**
> 1. Da: lanțul duce de la certificatul final, prin R11, la o rădăcină de încredere, iar numele se potrivește. 2. **Nu**: nepotrivire de nume — certificatul trebuie să conțină și `shop.ro` (majoritatea îl conțin, ca al doilea nume). 3. Adesea nu: lanțul este incomplet; unele browsere preiau sau își amintesc R11, altele eșuează — o eroare clasică de configurare. 4. **Nu**: expirat. 5. **Nu**: cheia lui nu duce la nicio rădăcină de încredere. Browserul afișează un avertisment pe toată pagina; utilizatorul care trece totuși mai departe acceptă exact atacul de tip om-la-mijloc pe care TLS există ca să-l prevină.

E. TELNET, chei de host SSH și tuneluri

Problema 7

O captură a unei sesiuni TELNET conține, de la server, octeții `FF FD 18 FF FD 20 FF FB 01 FF FB 03`, iar mai târziu, de la client, caracterele `s t u d e n t \r \n p a 5 5 w 0 r d \r \n`.

1. Decodificați octeții de opțiuni ai serverului (0x18 = 24 este TERMINAL-TYPE, 0x20 = 32 este TERMINAL-SPEED).
2. De ce nu a apărut parola pe ecranul utilizatorului, deși apare în captură?

3. Dați alte două lucruri pe care le-ar putea face un atacator de pe traseu, în afară de citirea parolei.

> **Soluție**
> 1. IAC DO TERMINAL-TYPE, IAC DO TERMINAL-SPEED (serverul îi cere clientului să le raporteze), IAC WILL ECHO (serverul va face ecou), IAC WILL SUPPRESS-GO-AHEAD.
> 2. După WILL ECHO, clientul nu mai face ecou local; serverul alege să nu facă ecou în timpul parolei. Ascunderea caracterelor pe ecran nu schimbă nimic pe fir: ele circulă în clar.
> 3. Să citească tot restul a ceea ce se tastează și se afișează în sesiune; să injecteze comenzi în fluxul TCP (deturnarea sesiunii — session hijacking), deoarece nimic nu autentifică datele; să refolosească mai târziu parola pe alte mașini unde utilizatorul a ales aceeași parolă.

Problema 8

V-ați autentificat pe `lab.cs.example.ro` cu SSH timp de luni de zile. Astăzi `ssh` afișează „WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!” și refuză să se conecteze.

1. Dați două explicații posibile.
2. Ce ar trebui să faceți înainte de a vă conecta? Ce **nu** trebuie să faceți?
3. De ce vă protejează acest avertisment, deși nu este implicată nicio autoritate de certificare?

> **Soluție**
> 1. Serverul a fost reinstalat sau cheile sale de host au fost regenerate (nevinovat); sau numele duce acum către o altă mașină — printr-o modificare DNS, o rețea compromisă sau un om-la-mijloc.
> 2. Întrebați administratorii, pe un alt canal, dacă cheia s-a schimbat și care este noua ei amprentă; comparați-o cu amprenta afișată. Abia apoi eliminați linia veche cu `ssh-keygen -R lab.cs.example.ro` și conectați-vă. Nu ștergeți orbește liniile din `known_hosts` și nu dezactivați verificarea cheii de host: asta înseamnă acceptarea atacului pe care îl descrie avertismentul.
> 3. Încredere la prima utilizare: prima conexiune a stocat cheia reală. Un impostor ar avea nevoie de cheia privată de host a serverului pentru a produce o semnătură validă; fără ea, poate doar să prezinte o cheie diferită, pe care clientul o detectează.

Problema 9 — proiectare

Scrieți comanda SSH pentru fiecare nevoie. Aveți un cont pe `bastion.uni.ro`, accesibil din Internet pe portul 22.

1. O bază de date de la `db.internal:5432` este accesibilă doar din interiorul universității; doriți ca un client de baze de date de pe laptopul dumneavoastră să se conecteze la ea.
2. Rulați o aplicație web pe laptop, pe portul 3000, și doriți ca un coleg să o deschidă prin bastion, pe portul 9000.
3. Doriți ca browserul dumneavoastră să ajungă o vreme la orice site web accesibil doar din universitate.
4. În cazul 1, ce porțiuni ale traseului sunt criptate?

Soluție

1. `ssh -N -L 5432:db.internal:5432 you@bastion.uni.ro`, apoi îndreptați clientul către `localhost:5432` (`-N`: fără shell la distanță, doar redirecționare).
2. `ssh -N -R 9000:localhost:3000 you@bastion.uni.ro`; colegul deschide `http://bastion.uni.ro:9000/` (serverul trebuie să permită acest lucru, opțiunea GatewayPorts).
3. `ssh -N -D 1080 you@bastion.uni.ro` și configurați browserul să folosească proxy-ul SOCKS `localhost:1080`.
4. Laptop → bastion: în interiorul SSH, criptat. Bastion → db.internal: o conexiune TCP obișnuită în interiorul rețelei universității, protejată doar dacă protocolul bazei de date folosește el însuși TLS. Porțiunea de la clientul bazei de date la localhost:5432 nu părăsește niciodată laptopul.

F. Citirea FTP

Problema 10

Un client de la 10.0.0.5 vorbește cu un server FTP de la 198.51.100.20 (`->` de la client la server, `<-` de la server la client):

```
<- 220-Welcome to the course archive
<- 220 Anonymous access allowed
-> USER anonymous
<- 331 Please send your e-mail address as password
-> PASS student@uni.ro
<- 230 Login successful
-> TYPE I
<- 200 Switching to Binary mode
-> PORT 10,0,0,5,200,10
<- 200 PORT command successful
-> RETR lab4.zip
<- 150 Opening BINARY mode data connection for lab4.zip (52113 bytes)
<- 226 Transfer complete
-> QUIT
<- 221 Goodbye
```

1. De unde știe clientul că primul răspuns s-a încheiat? Ce îi spune prima cifră a codului 331?
2. Ce mod este folosit? Cine deschide conexiunea de date, către ce adresă și port și de pe ce port al serverului?
3. De unde știe clientul că lab4.zip a fost primit complet?
4. Clientul este acasă, în spatele unui router NAT; serverul este în Internet. Ce pas eșuează și cu ce răspuns? Cum ați modifica sesiunea?

Soluție

1. Un răspuns pe mai multe linii începe cu `220-` și se termină cu linia care începe cu `220 ` (codul și un spațiu). 3 = pozitiv intermediar: comanda a fost acceptată, este nevoie de mai multe informații — parola.
2. Modul activ (PORT): **serverul** se conectează la 10.0.0.5, portul 200 × 256 + 10 = **51 210**, de pe portul său 20.
3. În modul flux, serverul închide conexiunea de date la sfârșitul fișierului; clientul verifică și codul 226 pe conexiunea de control și poate compara cu cei 52 113 octeți anunțați în răspunsul 150.
4. 10.0.0.5 este privată și inaccesibilă din Internet; încercarea de conectare a serverului eșuează după RETR, iar serverul răspunde **425 Can't open data connection**. Folosiți modul pasiv: trimiteți PASV (sau EPSV) în loc de PORT și conectați-vă la adresa și portul din răspunsul 227. (Un router cu un gateway la nivel de aplicație pentru FTP ar putea rescrie comanda PORT și deschide o mapare temporară, dar acest lucru eșuează dacă conexiunea de control este criptată cu FTPS.)

Problema 11 — ASCII sau binar?

O imagine PNG de 10 000 de octeți conține 40 de octeți egali cu 0x0A și 12 octeți egali cu 0x0D. Este descărcată cu `TYPE A` de pe un server Linux pe un client Windows.

1. Ce trimite serverul pentru fiecare octet 0x0A și ce stochează clientul Windows?
2. Cât de lung este fișierul pe discul Windows și este imaginea utilizabilă?
3. Ce s-ar întâmpla în direcția opusă, la încărcarea fișierului Windows cu TYPE A pe un server Linux?

Soluție

1. În tipul ASCII, serverul își convertește terminatorul de linie (LF, 0x0A) în CR LF-ul rețelei (0x0D 0x0A); clientul Windows convertește CR LF în propriul CR LF, astfel încât fiecare 0x0A sosește ca 0x0D 0x0A.
2. 10 000 + 40 = **10 040 de octeți**; imaginea este coruptă (decodorul citește date fără sens după prima modificare).
3. Clientul și-ar trimite perechile CR LF ca CR LF; serverul Linux convertește fiecare CR LF în LF, eliminând octeți. Datele care se întâmplă să conțină 0x0D 0x0A sunt deteriorate. Doar `TYPE I` transferă octeții nemodificați.

Problema 12 — multe fișiere mici

Un utilizator descarcă **20 de fișiere de câte 5 KB** de pe un server; RTT = 40 ms; ignorați timpii de transmisie (5 KB este puțin) și întârzierile serverului. După autentificare, comparați:

1. FTP în mod pasiv, un fișier după altul: pentru fiecare fișier, PASV și răspunsul său, un handshake TCP pentru conexiunea de date, apoi RETR și datele.
2. HTTP/1.1 pe o singură conexiune persistentă, o cerere după alta.
3. HTTP/2, toate cele 20 de cereri trimise deodată.

> **Soluție**
>
> 1. Pentru fiecare fișier: PASV (1 RTT) + handshake-ul conexiunii de date (1 RTT) + RETR și datele (1 RTT) = 3 RTT = 120 ms; 20 de fișiere: **2,4 s**.
> 2. 1 RTT pentru fiecare fișier: **0,8 s**.
> 3. Aproximativ **1 RTT = 40 ms** pentru toate. Proiectarea FTP cu o conexiune pentru fiecare fișier, naturală în 1985, este costisitoare pentru multe fișiere mici — unul dintre motivele pentru care arhivele se descarcă de obicei ca un singur fișier comprimat.

Temă

- Rezolvați exercițiile 1–5 de la sfârșitul Cursului 4 fără să vă uitați la răspunsuri.
- Rulați `curl -sI https://www.google.com | grep -i alt-svc` și explicați ce anunță linia.
- Citiți RFC 959, secțiunea 3.4 (modurile de transmisie), și explicați cum permite modul bloc repornirea unui transfer după o eroare și de ce modul flux nu poate face acest lucru.

PARTEA C · SĂPTĂMÂNA 4

Laboratorul 4 (Linux): sesiuni, securitate și transfer de fișiere — cookie-uri, HTTPS, TELNET, SSH și FTP

Fișă de laborator de două ore (2 × 50 de minute) care însoțește Cursul 4

Săptămâna aceasta priviți protocoalele din Cursul 4 din ambele părți. Servere mici scrise pentru curs rulează pe propria mașină, astfel încât fiecare schimb este sub controlul dumneavoastră; instrumentele standard — curl, openssl, telnet, ssh, ftp, tcpdump — joacă rolul clientului și al observatorului; iar în ultima sarcină completați un client FTP în C care se bazează pe Laboratorul 3. Toate programele au fost testate pe Ubuntu 24.04 și sunt furnizate ca fișiere.

La sfârșitul laboratorului ar trebui să puteți

- urmări un cookie de sesiune și un GET condiționat cu curl;
- inspecta o conexiune TLS și un lanț de certificate și vedea numele serverului într-o captură;
- arăta că o autentificare de tip TELNET circulă în clar și decodifica octeții de negociere a opțiunilor;
- vă autentifica prin SSH folosind o pereche de chei, verifica amprenta unei chei de host și construi un tunel local;
- observa conexiunile de control și de date ale FTP în mod pasiv și activ;
- citi, rula și extinde un client FTP în mod pasiv scris cu socket-uri.

Înainte de a începe

- **Mediul de lucru.** Mașina virtuală a cursului sau orice Linux. Instalați instrumentele o singură dată (pe Ubuntu, `python3-pyftpdlib` se află în depozitul universe; în altă parte folosiți `pip install pyftpdlib`):

```
sudo apt install -y build-essential curl openssl telnet tcpdump \
    openssh-client openssh-server inetutils-ftp python3-pyftpdlib
```

- **Fișiere.** Dezarhivați arhiva acestui laborator în `lab4/`: `linux/` (netio.h, ftp_client.c, Makefile), `python/` (web_server.py, login_server.py, ftp_client.py) și `windows/`. Compilați cu `make -C linux`.
- **Terminale.** Majoritatea sarcinilor au nevoie de trei: un server, un client și un observator care rulează `ss` sau `tcpdump`. `tcpdump -i lo -A` afișează ca text pachetele de pe interfața loopback.
- **Etică.** Capturați doar propriul trafic, pe propria mașină. Nu încercați niciodată parolele pe care le vedeți pe contul altcuiva.
- **Ce predați.** Un scurt raport cu răspunsurile la **Q1–Q16** și ieșirile care le susțin; fișierul `ftp_client.c` extins de dumneavoastră.

Plan

Pas	Activitate	Timp
1	Stare și cache: cookie-uri și GET-ul condiționat	15 min

Pas	Activitate	Timp
2	HTTPS: o conexiune TLS, un certificat, numele vizibil	15 min
3	Autentificare la distanță în clar: o sesiune de tip TELNET	15 min
	Pauză	10 min
4	SSH: chei de host și autentificare cu cheie publică	15 min
5	Tuneluri SSH și copierea fișierelor	10 min
6	FTP cu instrumente standard: două conexiuni, două moduri	15 min
7	Un client FTP propriu	15 min

Sarcina 1 — Stare și cache

`web_server.py` servește o pagină care vă numără vizitele cu ajutorul unui cookie de sesiune și un fișier cu ETag și `Cache-Control: max-age=30`; `POST /touch` modifică fișierul. Afișează antetele fiecărei cereri pe care o primește.

```
"""web_server.py - un mic server HTTP/1.1 pentru Laboratorul 4: cookie-uri și cache.
utilizare: python3 web_server.py [port]    (ascultă pe 127.0.0.1, implicit 8000)
  GET  /            o pagină care vă numără vizitele cu un cookie de sesiune
  GET  /notes.txt   un fișier cu ETag, Last-Modified, Cache-Control max-age
  POST /touch       modifică notes.txt, astfel încât ETag-ul său se schimbă"""
import hashlib, secrets, sys, time
from email.utils import formatdate
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

sessions = {}        # id sesiune -> vizite: starea se află pe server
notes = {"text": "Lecture 4 notes, version 1\n", "mtime": time.time()}


class Handler(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"               # conexiuni persistente

    def send(self, code, body=b"", headers=()):
        self.send_response(code)
        for k, v in headers:
            self.send_header(k, v)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        if body and self.command != "HEAD":
            self.wfile.write(body)

    def do_GET(self):
        print("   request headers:", dict(self.headers))
        if self.path == "/":
            cookies = {}
            for part in self.headers.get("Cookie", "").split(";"):
                if "=" in part:
                    k, v = part.strip().split("=", 1)
                    cookies[k] = v
            sid, extra = cookies.get("session"), []
            if sid not in sessions:     # prima vizită: stare nouă, cheie nouă
                sid = secrets.token_hex(8)
                sessions[sid] = 0
                extra.append(("Set-Cookie", f"session={sid}; Path=/; "
                              "HttpOnly; SameSite=Lax"))
            sessions[sid] += 1
            body = (f"<p>Hello! This is visit number {sessions[sid]} "
```

```
                    f"of session {sid}.</p>\n").encode()
            self.send(200, body,
                      [("Content-Type", "text/html; charset=utf-8"), *extra])
        elif self.path == "/notes.txt":
            body = notes["text"].encode()
            etag = '"' + hashlib.sha1(body).hexdigest()[:12] + '"'
            if self.headers.get("If-None-Match") == etag:
                # GET condiționat, iar copia este încă validă: fără corp
                self.send(304, headers=[("ETag", etag),
                                        ("Cache-Control", "max-age=30")])
                return
            modified = formatdate(notes["mtime"], usegmt=True)
            self.send(200, body, [("Content-Type", "text/plain"),
                                  ("ETag", etag),
                                  ("Last-Modified", modified),
                                  ("Cache-Control", "max-age=30")])
        else:
            self.send(404, b"not found\n", [("Content-Type", "text/plain")])

    do_HEAD = do_GET

    def do_POST(self):
        self.rfile.read(int(self.headers.get("Content-Length", 0)))
        if self.path == "/touch":
            n = int(notes["text"].split()[-1]) + 1
            notes.update(text=f"Lecture 4 notes, version {n}\n",
                         mtime=time.time())
            self.send(200, b"notes.txt changed\n",
                      [("Content-Type", "text/plain")])
        else:
            self.send(404, b"not found\n", [("Content-Type", "text/plain")])


port = int(sys.argv[1]) if len(sys.argv) > 1 else 8000
print(f"serving on http://127.0.0.1:{port}/")
ThreadingHTTPServer(("127.0.0.1", port), Handler).serve_forever()
```

```
python3 python/web_server.py 8000                                    # terminalul A
# terminalul B: căutați Set-Cookie
curl -i http://127.0.0.1:8000/
# rulați de 3 ori: fișierul jar păstrează cookie-ul
curl -c jar.txt -b jar.txt http://127.0.0.1:8000/
cat jar.txt
curl -i http://127.0.0.1:8000/notes.txt                              # notați ETag-ul
curl -i -H 'If-None-Match: "<ETag-ul>"' http://127.0.0.1:8000/notes.txt
# apoi repetați GET-ul condiționat
curl -X POST http://127.0.0.1:8000/touch
```

- **Q1.** Arătați antetul `Set-Cookie` al primului răspuns și antetul `Cookie` afișat de server pentru cererile următoare. Unde este stocat contorul de vizite — în cookie sau pe server? Ce se întâmplă dacă ștergeți `jar.txt`?
- **Q2.** Arătați linia de stare și lungimea corpului pentru GET-ul condiționat înainte și după `/touch`. Explicați diferența cu termenii din Cursul 4, secțiunea 3.1.

Sarcina 2 — HTTPS

```
# versiunea TLS, cifrul, certificatul
curl -v https://example.org/ -o /dev/null 2>&1 | grep '^\*'
# versiunea de protocol folosită efectiv
curl -sI --http2 https://example.org/ | head -1
S=example.org
openssl s_client -connect $S:443 -servername $S -brief < /dev/null
# lanțul: subiectul (s:) și emitentul (i:) fiecărui certificat
openssl s_client -connect $S:443 -servername $S -showcerts < /dev/null \
    2>/dev/null | grep -E '^ *[0-9] s:|^ *i:'
# apoi repetați primul curl
sudo tcpdump -i any -n -A -c 6 'tcp port 443' &
```

- **Q3.** Ce versiune TLS, ce suită de cifruri și ce grup de schimb de chei au fost negociate? Ce versiune HTTP? Scrieți lanțul de certificate de la certificatul serverului până la rădăcină, cu emitentul fiecăruia.
- **Q4.** În captură, găsiți ClientHello și textul lizibil din interiorul lui. Ce nume apare și de ce trebuie să circule în clar? Puteți găsi calea `/` sau vreun antet HTTP?

Sarcina 3 — Autentificare la distanță în clar

`login_server.py` imită un vechi serviciu TELNET: cere un nume de utilizator și o parolă și trimite înapoi liniile primite (ecou). Înainte de parolă trimite octeții TELNET IAC WILL ECHO, astfel încât un client TELNET să nu mai afișeze ceea ce tastați.

```
"""login_server.py - un serviciu-jucărie de autentificare la distanță care trimite totul
în clar, ca TELNET (Laboratorul 4).
utilizare: python3 login_server.py [port]   (ascultă pe 127.0.0.1, implicit 2323)
conectare: telnet 127.0.0.1 2323   (nc 127.0.0.1 2323 arată octeții bruți)"""
import socket, sys, threading

IAC, WILL, WONT, DO, DONT = 255, 251, 252, 253, 254   # comenzi TELNET
ECHO = 1                                               # opțiunea TELNET 1


def read_line(conn):
    """Citește o linie, eliminând negocierile TELNET (IAC comandă opțiune)."""
    data = bytearray()
    while True:
        b = conn.recv(1)
        if not b:
            return None
        if b[0] == IAC:             # IAC DO/DONT/WILL/WONT opțiune: o sărim
            conn.recv(2)
            continue
        if b == b"\n":
            return data.decode(errors="replace").rstrip("\r")
        data += b


def session(conn, addr):
    with conn:
        conn.sendall(b"Toy login service (everything travels in clear)\r\n"
                     b"login: ")
        user = read_line(conn)
        # „Eu voi face ecou”: un client TELNET nu mai afișează ce tastează utilizatorul
        conn.sendall(b"password: " + bytes([IAC, WILL, ECHO]))
        password = read_line(conn)
        conn.sendall(bytes([IAC, WONT, ECHO]) + b"\r\n")   # ecou normal
        if user is None or password is None:
            return
```

```
        print(f"{addr[0]}:{addr[1]} logged in as {user!r}, "
              f"password {password!r}")
        conn.sendall(f"Welcome, {user}. Type a line, or 'quit'.\r\n".encode())
        while (line := read_line(conn)) not in (None, "quit"):
            conn.sendall(f"you said: {line}\r\n".encode())
        conn.sendall(b"bye\r\n")


port = int(sys.argv[1]) if len(sys.argv) > 1 else 2323
srv = socket.create_server(("127.0.0.1", port))
print(f"toy login service on 127.0.0.1:{port}")
while True:
    c, a = srv.accept()
    threading.Thread(target=session, args=(c, a), daemon=True).start()
```

```
python3 python/login_server.py 2323                                  # terminalul A
sudo tcpdump -i lo -A -n 'tcp port 2323'                              # terminalul C
# terminalul B: autentificați-vă, tastați o linie, ieșiți (quit)
telnet 127.0.0.1 2323
# din nou B: octeții bruți, fără client TELNET
nc 127.0.0.1 2323 | od -c | head
```

- **Q5.** Copiați din captură pachetele care conțin numele de utilizator și parola. A apărut parola pe ecranul dumneavoastră? Explicați ambele observații.
- **Q6.** În ieșirea `od -c`, găsiți octeții afișați ca `377 373 001` și `377 374 001` (în octal). Traduceți-i în hexazecimal și în comenzi TELNET.

Sarcina 4 — SSH: chei de host și autentificare cu cheie publică

```
sudo systemctl start ssh                          # serverul SSH de pe această mașină
# amprenta cheii de host a serverului
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
ssh -v localhost exit 2>&1 \
    | grep -E 'version|kex: algorithm|host key|Authentications|Authenticated'
# perechea dumneavoastră de chei (alegeți o frază de acces)
ssh-keygen -t ed25519
# instalează cheia publică în ~/.ssh/authorized_keys
ssh-copy-id localhost
# acum fără parola contului
ssh localhost 'hostname; whoami; uptime'
```

- **Q7.** La prima conexiune, ce amprentă a afișat ssh și se potrivește ea cu cea a serverului? În ce fișier a fost stocată cheia? Din liniile `-v`, scrieți versiunea protocolului, algoritmul de schimb de chei, tipul cheii de host și metodele de autentificare oferite.
- **Q8.** Uitați-vă la `~/.ssh/id_ed25519.pub` și la `~/.ssh/authorized_keys`. Ce conține fiecare, ce fișier nu părăsește niciodată mașina dumneavoastră și de ce este autentificarea acum sigură chiar dacă serverul ar fi compromis?

Sarcina 5 — Tuneluri SSH și copierea fișierelor

```
# terminalul D: portul local 2324 -> ssh -> 2323
ssh -N -L 2324:127.0.0.1:2323 localhost
# terminalul B: autentificați-vă din nou, prin tunel
telnet 127.0.0.1 2324
# terminalul C, apoi 'tcp port 2323' și 'tcp port 2324'
sudo tcpdump -i lo -A -n 'tcp port 22'
# în sftp: ls /tmp, get, bye
scp python/web_server.py localhost:/tmp/  &&  sftp localhost
```

- **Q9.** Care dintre cele trei capturi arată parola și care nu? Pe o singură mașină toate porțiunile sunt locale; desenați cele trei porțiuni așa cum ar fi dacă serverul SSH ar fi un gateway în altă clădire și spuneți care dintre ele traversează rețeaua nesigură.
- **Q10.** `sftp` și `ftp` sună asemănător. Ce port și ce protocol folosește sftp și câte conexiuni TCP deschide?

Sarcina 6 — FTP cu instrumente standard

```
mkdir -p ~/ftproot && cp -r python ~/ftproot/
head -c 200000 /dev/urandom > ~/ftproot/data.bin           # un fișier binar
python3 -m pyftpdlib -p 2121 -d ~/ftproot -u student -P secret        # terminalul A
sudo tcpdump -i lo -A -n 'tcp port 2121'                              # terminalul C
# terminalul B; apoi tastați:
ftp -d -v -p 127.0.0.1 2121
    user student secret | ls | binary | get data.bin | passive | ls | quit
# terminalul D, în timpul unui transfer
watch -n 0.5 'ss -tn | grep 127.0.0.1'
```

- **Q11.** Copiați comenzile și răspunsurile sesiunii (liniile `--->` și răspunsurile de trei cifre). Găsiți răspunsul la PASV și calculați portul de date; găsiți comanda PORT trimisă după ce `passive` a dezactivat modul pasiv și calculați portul pe care a ascultat clientul. Cine a deschis fiecare conexiune de date?
- **Q12.** Câte conexiuni TCP a folosit sesiunea în total? Care dintre ele a transportat parola și a fost aceasta lizibilă în captură? Rulați `ls` și `get` în timp ce urmăriți `ss`: ce trăiește cât sesiunea și ce trăiește doar pe durata unui transfer?

Sarcina 7 — Un client FTP propriu

Funcțiile ajutătoare din Laboratorul 3 primesc două funcții noi: `recv_line` citește o linie terminată cu CR LF, așa cum au nevoie răspunsurile FTP, iar `tcp_connect` încapsulează `getaddrinfo` și `connect`.

```
/* Citește o linie terminată cu LF (CR LF este eliminat). Întoarce lungimea ei,
   0 dacă partenerul a închis înainte de orice octet, -1 la eroare sau linie prea lungă.
*/
static inline long recv_line(int fd, char *buf, size_t cap) {
    size_t n = 0;
    for (;;) {
        char ch;
        ssize_t r = recv(fd, &ch, 1, 0);
        if (r < 0) { if (errno == EINTR) continue; return -1; }
        if (r == 0) { if (n == 0) return 0; break; }
        if (ch == '\n') break;
        if (n + 1 >= cap) return -1;
        buf[n++] = ch;
    }
    if (n > 0 && buf[n - 1] == '\r') n--;
    buf[n] = '\0';
    return (long)n;
}

/* Conectează un socket TCP la host:port, încercând fiecare adresă întoarsă
   de getaddrinfo. */
static inline int tcp_connect(const char *host, const char *port) {
    struct addrinfo hints = {0}, *res, *ai;
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    if (getaddrinfo(host, port, &hints, &res) != 0) return -1;
    int fd = -1;
    for (ai = res; ai; ai = ai->ai_next) {
```

```
        fd = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
        if (fd < 0) continue;
        if (connect(fd, ai->ai_addr, ai->ai_addrlen) == 0) break;
        close(fd); fd = -1;
    }
    freeaddrinfo(res);
    return fd;
}
```

Clientul se autentifică, alege tipul de transfer, cere modul pasiv, decodifică răspunsul 227, deschide conexiunea de date și copiază tot ce primește până când serverul o închide — sfârșitul fișierului în modul flux.

```
/* ftp_client.c - un client FTP minimal în mod pasiv (Rețele de calculatoare, Laboratorul
4)
   utilizare: ./ftp_client host port user password ls [dir]
              ./ftp_client host port user password get file                  */
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include "netio.h"

static int ctrl;                            /* conexiunea de control */

/* Citește un răspuns, eventual pe mai multe linii ("150-..." ... "150 ..."),
   îl afișează și întoarce codul său de trei cifre. */
static int reply(void) {
    char line[1024], first[4] = "";
    for (;;) {
        if (recv_line(ctrl, line, sizeof line) <= 0) {
            fprintf(stderr, "control connection lost\n");
            exit(1);
        }
        printf("<- %s\n", line);
        if (first[0] == '\0') memcpy(first, line, 3);  /* codul liniei 1 */
        if (strncmp(line, first, 3) == 0 && line[3] == ' ')
            return atoi(first);                        /* "ddd " îl încheie */
    }
}

/* Trimite o comandă terminată cu CR LF (parola nu este afișată),
   apoi citește și întoarce codul de răspuns. */
static int command(const char *fmt, ...) {
    char cmd[512];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(cmd, sizeof cmd - 2, fmt, ap);
    va_end(ap);
    printf("-> %s\n", strncmp(cmd, "PASS ", 5) == 0 ? "PASS ****" : cmd);
    strcat(cmd, "\r\n");
    if (send_all(ctrl, cmd, strlen(cmd)) < 0) { perror("send"); exit(1); }
    return reply();
}

int main(int argc, char *argv[]) {
    if (argc < 6) {
        fprintf(stderr, "usage: %s host port user password ls [dir]"
                        " | get file\n", argv[0]);
        return 1;
    }
    int get = strcmp(argv[5], "get") == 0;
    if ((ctrl = tcp_connect(argv[1], argv[2])) < 0) {
        fprintf(stderr, "cannot connect\n");
        return 1;
    }
```

```
    if (reply() != 220) return 1;                       /* serviciu pregătit */
    if (command("USER %s", argv[3]) == 331 &&
        command("PASS %s", argv[4]) != 230) return 1;  /* autentificat? */
    command(get ? "TYPE I" : "TYPE A");          /* tip imagine sau ASCII */

    /* PASV: răspunsul este 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2) */
    printf("-> PASV\n");
    send_all(ctrl, "PASV\r\n", 6);
    char line[1024];
    int h1, h2, h3, h4, p1, p2;
    do {
        if (recv_line(ctrl, line, sizeof line) <= 0) return 1;
        printf("<- %s\n", line);
    } while (strncmp(line, "227 ", 4) != 0);
    char *q = strchr(line, '(');
    if (!q || sscanf(q, "(%d,%d,%d,%d,%d,%d)",
                     &h1, &h2, &h3, &h4, &p1, &p2) != 6) return 1;
    char port[8];
    snprintf(port, sizeof port, "%d", p1 * 256 + p2);
    printf("   data connection: %d.%d.%d.%d port %d*256+%d = %s\n",
           h1, h2, h3, h4, p1, p2, port);
    int data = tcp_connect(argv[1], port);   /* același host, portul din PASV */
    if (data < 0) {
        fprintf(stderr, "cannot open data connection\n");
        return 1;
    }

    int code = get ? command("RETR %s", argv[6])
             : argc > 6 ? command("LIST %s", argv[6]) : command("LIST");
    if (code != 125 && code != 150) {            /* de ex. 550 fișier inexistent */
        close(data);
        command("QUIT");
        return 1;
    }
    FILE *out = get ? fopen(argv[6], "wb") : stdout;
    if (!out) { perror("fopen"); return 1; }
    char buf[4096];
    ssize_t n;
    long total = 0;
    while ((n = recv(data, buf, sizeof buf, 0)) > 0) {
        fwrite(buf, 1, (size_t)n, out);
        total += n;
    }
    close(data);              /* modul flux: sfârșit de fișier = conexiune închisă */
    if (get) {
        fclose(out);
        printf("   %ld bytes written to %s\n", total, argv[6]);
    }
    reply();                                       /* 226 transfer încheiat */
    command("QUIT");                               /* 221 la revedere */
    close(ctrl);
    return 0;
}
```

```
make -C linux
./linux/ftp_client 127.0.0.1 2121 student secret ls
./linux/ftp_client 127.0.0.1 2121 student secret get data.bin
cmp data.bin ~/ftproot/data.bin && echo identical
./linux/ftp_client 127.0.0.1 2121 student secret get missing.txt
echo exit code $?
```

- **Q13.** Explicați cum recunoaște `reply()` sfârșitul unui răspuns pe mai multe linii. Trimiteți FEAT după autentificare (adăugați o linie `command("FEAT")`) și arătați un răspuns pe mai multe linii tratat de funcția dumneavoastră.

- **Q14.** De ce trebuie deschisă conexiunea de date **înainte** de trimiterea RETR și de ce citește programul răspunsul final 226 doar după ce conexiunea de date s-a închis?
- **Q15.** Înlocuiți PASV cu EPSV: răspunsul este `229 Entering Extended Passive Mode (|||port|)`. Modificați analiza (parsing-ul) și testați. Ce nu mai conține EPSV și de ce îl face acest lucru să funcționeze cu IPv6 și prin NAT?
- **Q16.** Adăugați o comandă `put file`: TYPE I, PASV, STOR name, trimiteți octeții fișierului pe conexiunea de date și închideți-o, apoi citiți 226. (Porniți serverul cu `-w` pentru a permite scrierea.) Arătați o încărcare reușită și un `cmp`.

Aceeași sesiune în Python

Modulul `ftplib` din Python implementează protocolul în locul dumneavoastră; cu depanarea activată, puteți compara dialogul său linie cu linie cu clientul dumneavoastră în C.

```
"""ftp_client.py - aceeași sesiune în mod pasiv cu ftplib din Python.
utilizare: python3 ftp_client.py host port user password ls [dir] | get file"""
import sys
from ftplib import FTP

host, port, user, password, what = sys.argv[1:6]
arg = sys.argv[6] if len(sys.argv) > 6 else ""
ftp = FTP()
ftp.set_debuglevel(1)        # afișează fiecare comandă (*cmd*) și răspuns (*resp*)
ftp.connect(host, int(port))
ftp.login(user, password)
ftp.set_pasv(True)           # pasiv: clientul deschide conexiunea de date
if what == "ls":
    ftp.retrlines("LIST " + arg if arg else "LIST")          # TYPE A, LIST
else:
    with open(arg, "wb") as f:
        ftp.retrbinary("RETR " + arg, f.write)               # TYPE I, RETR
ftp.quit()
```

Dacă ceva nu funcționează

Simptom	Cauza probabilă și ce să încercați
`ssh: connect to host localhost port 22: Connection refused`	serverul nu rulează: `sudo systemctl start ssh`; verificați cu `ss -tln | grep :22`
`Permission denied (publickey)` după ssh-copy-id	permisiuni greșite: `chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys`
`python3 -m pyftpdlib`: -u nerecunoscut	o versiune veche: `pip install --user pyftpdlib` sau folosiți accesul anonim fără -u/-P
`425 Can't open data connection`	mod activ blocat de un firewall: folosiți modul pasiv (`passive` în ftp, `-p`)
openssl sau curl nu pot ajunge la example.org	rețeaua laboratorului îl blochează: folosiți un alt site HTTPS public indicat de cadrul didactic
tcpdump nu arată nimic	interfață greșită: folosiți `-i lo` pentru 127.0.0.1, `-i any` pentru traficul extern

PARTEA D · SĂPTĂMÂNA 4

Laboratorul 4 (Windows): aceleași protocoale cu instrumente Windows și Winsock

Fișă de laborator de două ore (2 × 50 de minute) care însoțește Cursul 4

Această fișă urmează fișa Linux sarcină cu sarcină, astfel încât răspunsurile să poată fi comparate. Windows 10 și 11 includ aproape tot ce este necesar: `curl.exe`, clientul OpenSSH (`ssh`, `ssh-keygen`, `scp`, `sftp`) și PowerShell, ale cărui clase .NET vă permit să deschideți conexiuni TLS și să inspectați certificate. Capturile se fac cu Wireshark, al cărui driver Npcap poate captura și traficul loopback. Serverele cursului sunt aceleași programe Python ca pe Linux.

La sfârșitul laboratorului ar trebui să puteți

- urmări un cookie de sesiune și un GET condiționat cu curl.exe și PowerShell;
- inspecta din PowerShell o conexiune TLS și lanțul ei de certificate și găsi numele serverului în Wireshark;
- arăta că o autentificare de tip TELNET circulă în clar;
- vă autentifica pe o mașină Linux cu o cheie SSH din Windows și trimite o pagină web printr-un tunel SSH;
- observa modurile activ și pasiv ale FTP cu ftp.exe și curl.exe;
- construi și extinde clientul FTP cu Winsock.

Înainte de a începe

- **Python 3** de pe python.org (bifați „Add to PATH”); apoi `py -m pip install pyftpdlib`.
- **Un compilator C**, ca în Laboratorul 3: MinGW-w64 (`gcc ... -lws2_32`) sau Visual Studio Build Tools (`cl ... ws2_32.lib`).
- **Wireshark** cu Npcap; în timpul instalării păstrați bifată opțiunea „Support loopback traffic capture”. Capturați pe *Adapter for loopback traffic capture* pentru 127.0.0.1.
- **Un server SSH** pe care să vă autentificați: mașina virtuală Linux a cursului, un server de laborator sau componenta opțională proprie a Windows (*(admin)* `Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0`, apoi `Start-Service sshd`). Mai jos este numit `user@server`.
- **Clientul TELNET** (opțional): *(admin)* `dism /online /Enable-Feature /FeatureName:TelnetClient`. Fără el, folosiți fragmentul PowerShell din Sarcina 3.
- **Fișiere.** Dezarhivați arhiva în `lab4\`; folderul Windows conține `netio_win.h` și `ftp_client_win.c`; serverele Python se află în `python\`.

Plan

Pas	Activitate	Timp
1	Stare și cache: cookie-uri și GET-ul condiționat	15 min
2	HTTPS: o conexiune TLS, un certificat, numele vizibil	15 min
3	Autentificare la distanță în clar: o sesiune de tip TELNET	15 min

Pas	Activitate	Timp
	Pauză	10 min
4	SSH din Windows: chei de host și autentificare cu cheie publică	15 min
5	Un tunel SSH către o pagină web de pe server	10 min
6	FTP cu ftp.exe și curl.exe: activ și pasiv	15 min
7	Clientul FTP cu Winsock	15 min

Sarcina 1 — Stare și cache

```
py python\web_server.py 8000                                          # fereastra A
# fereastra B: Set-Cookie
curl.exe -i http://127.0.0.1:8000/
curl.exe -c jar.txt -b jar.txt http://127.0.0.1:8000/                 # de trei ori
$u = 'http://127.0.0.1:8000/'
$r = Invoke-WebRequest $u -SessionVariable web -UseBasicParsing
$web.Cookies.GetCookies($u) | Format-Table Name, Value, HttpOnly
(Invoke-WebRequest $u -WebSession $web -UseBasicParsing).Content
curl.exe --etag-save etag.txt -o notes.txt http://127.0.0.1:8000/notes.txt
# trimite If-None-Match
curl.exe -i --etag-compare etag.txt http://127.0.0.1:8000/notes.txt
# apoi repetați ultima comandă
curl.exe -X POST http://127.0.0.1:8000/touch
```

- **Q1.** Arătați antetul `Set-Cookie` și valorile Cookie pe care PowerShell le păstrează în `$web`. Unde este stocat contorul? Ce se întâmplă cu un nou `-SessionVariable`?
- **Q2.** Arătați linia de stare a cererii condiționate înainte și după `/touch`, precum și antetul `If-None-Match` afișat de server. Explicați diferența.

Sarcina 2 — HTTPS

PowerShell poate deschide el însuși o conexiune TLS cu clasa .NET `SslStream` și poate arăta ce s-a negociat și ce certificate au fost prezentate:

```
$tcp = New-Object Net.Sockets.TcpClient('example.org', 443)
$tls = New-Object Net.Security.SslStream($tcp.GetStream())
# handshake-ul; argumentul devine SNI
$tls.AuthenticateAsClient('example.org')
$tls.SslProtocol                                      # Tls12 sau Tls13
$X509 = 'Security.Cryptography.X509Certificates'
$cert = New-Object "$X509.X509Certificate2" $tls.RemoteCertificate
$cert | Format-List Subject, Issuer, NotBefore, NotAfter, Thumbprint
$chain = New-Object "$X509.X509Chain"
[void]$chain.Build($cert)
$chain.ChainElements | ForEach-Object { $_.Certificate.Subject }
$tls.Close(); $tcp.Close()
curl.exe -sI --http2 https://example.org/                 # versiunea HTTP folosită
```

Porniți o captură Wireshark pe adaptorul de rețea, cu filtrul de afișare `tls.handshake.type == 1` (ClientHello), și repetați conexiunea.

- **Q3.** Ce versiune TLS a fost negociată și ce versiune HTTP? Scrieți lanțul de la certificatul serverului până la rădăcină. Deschideți `certlm.msc` → Trusted Root Certification Authorities și găsiți acolo rădăcina.
- **Q4.** În ClientHello, extindeți *Extension: server_name*. Ce conține și de ce nu este criptat? Puteți găsi calea URL sau vreun antet HTTP undeva în captură?

Sarcina 3 — Autentificare la distanță în clar

Porniți `login_server.py` (listingul se află în fișa Linux, Sarcina 3) și o captură Wireshark pe adaptorul loopback cu filtrul `tcp.port == 2323`. Autentificați-vă cu clientul TELNET sau — fără el — citiți octeții bruți din PowerShell:

```
py python\login_server.py 2323                          # fereastra A
# fereastra B, dacă clientul este instalat
telnet 127.0.0.1 2323
$c = New-Object Net.Sockets.TcpClient('127.0.0.1', 2323); $s = $c.GetStream()
$w = New-Object IO.StreamWriter($s); $w.AutoFlush = $true; $w.NewLine = "`r`n"
$w.WriteLine('alice'); $w.WriteLine('secret123'); Start-Sleep 1
$b = New-Object byte[] 512; $n = $s.Read($b, 0, 512)
($b[0..($n-1)] | ForEach-Object { $_.ToString('X2') }) -join ' '; $c.Close()
```

- **Q5.** În Wireshark folosiți *Follow → TCP Stream*. Copiați liniile cu numele de utilizator și parola. A apărut parola pe ecran în clientul TELNET? Explicați.
- **Q6.** În ieșirea hexazecimală găsiți `FF FB 01` și `FF FC 01`. Ce comenzi TELNET sunt acestea și de ce o trimite serverul pe prima chiar înainte de a cere parola?

Sarcina 4 — SSH din Windows

```
ssh -v user@server exit 2>&1 |
    Select-String 'version|kex: algorithm|host key|Authentications|Authenticated'
Get-Content $HOME\.ssh\known_hosts                      # cheia de host stocată
# creează $HOME\.ssh\id_ed25519 și .pub
ssh-keygen -t ed25519
Get-Content $HOME\.ssh\id_ed25519.pub |
    ssh user@server "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"
ssh user@server 'hostname; whoami; uptime'                # cu cheia
```

Pe server, `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub` afișează amprenta cheii de host, pentru a o compara cu cea arătată la prima conexiune. Windows nu are `ssh-copy-id`; pipeline-ul de mai sus îi face treaba.

- **Q7.** Ce amprentă a afișat ssh la prima conexiune și se potrivește ea cu cea a serverului? Scrieți versiunea protocolului, algoritmul de schimb de chei, tipul cheii de host și metodele de autentificare din ieșirea `-v`.
- **Q8.** Ce conține `id_ed25519.pub`, ce s-a adăugat pe server și ce fișier nu trebuie să părăsească niciodată PC-ul dumneavoastră? De ce este cheia privată protejată printr-o frază de acces?

Sarcina 5 — Un tunel SSH către o pagină web de pe server

Copiați `web_server.py` pe server și porniți-l acolo; ascultă doar pe adresa 127.0.0.1 proprie a serverului, deci nu poate fi accesat direct de pe PC-ul dumneavoastră. Apoi construiți un tunel către el:

```
scp python\web_server.py user@server:/tmp/
# fereastra A (lăsați-o să ruleze)
ssh user@server 'python3 /tmp/web_server.py 8000'
ssh -N -L 8080:127.0.0.1:8000 user@server                  # fereastra B: tunelul
# fereastra C: prin tunel
curl.exe -i http://localhost:8080/notes.txt
# ls /tmp, get /tmp/web_server.py, bye
sftp user@server
```

- **Q9.** Capturați pe adaptorul de rețea real în timpul comenzii `curl.exe`. Ce porturi vedeți și puteți citi cererea HTTP? Ce porțiune a traseului transportă textul HTTP în clar și pe ce mașină?

- **Q10.** Ce protocol și ce port folosește `sftp`? Câte conexiuni TCP a deschis, comparativ cu o sesiune FTP?

Sarcina 6 — FTP cu ftp.exe și curl.exe

```
mkdir $HOME\ftproot; Copy-Item python\*.py $HOME\ftproot\
py -m pyftpdlib -p 2121 -d $HOME\ftproot -u student -P secret      # fereastra A
# fereastra B; în interiorul ftp.exe:
ftp
    debug   open 127.0.0.1 2121   student   secret   ls
    binary   get web_server.py   bye                    # câte una pe linie
# listare pasivă (EPSV)
curl.exe -v -u student:secret ftp://127.0.0.1:2121/
# mod activ (PORT/EPRT)
curl.exe -v -u student:secret --ftp-port - ftp://127.0.0.1:2121/
# fereastra C, în timpul unei sesiuni
netstat -ano | findstr 2121
```

Programul `ftp.exe` integrat în Windows suportă **doar modul activ**: pentru fiecare listare și transfer trimite PORT, iar serverul se conectează înapoi la el. Prima dată, Windows Firewall poate întreba dacă `ftp.exe` are voie să accepte conexiuni de intrare — această întrebare *este* chiar problema modului activ din Cursul 4. Cu un server extern, modul activ printr-un router NAT eșuează de obicei.

- **Q11.** Din ieșirea de depanare a ftp.exe, copiați o comandă PORT și calculați portul. Din `curl.exe -v`, copiați schimbul EPSV și portul. Cine a deschis conexiunea de date în fiecare caz?
- **Q12.** Câte conexiuni a folosit o sesiune ftp.exe cu `ls` și `get`? Care a transportat parola și este aceasta lizibilă într-o captură loopback (`tcp.port == 2121`)?

Sarcina 7 — Clientul FTP cu Winsock

`netio_win.h` adaugă `recv_line` și `tcp_connect` la funcțiile ajutătoare din Laboratorul 3, exact ca pe Linux. Programul diferă de versiunea Linux doar prin detaliile Winsock pe care le cunoașteți: `WSAStartup`, `SOCKET`, `closesocket`, `WSAGetLastError`.

```
/* ftp_client_win.c - un client FTP minimal în mod pasiv, cu Winsock
   (Rețele de calculatoare, Laboratorul 4)
   utilizare: ftp_client_win host port user password ls [dir]
              ftp_client_win host port user password get file              */
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include "netio_win.h"

static SOCKET ctrl;                          /* conexiunea de control */

/* Citește un răspuns, eventual pe mai multe linii ("150-..." ... "150 ..."),
   îl afișează și întoarce codul său de trei cifre. */
static int reply(void) {
    char line[1024], first[4] = "";
    for (;;) {
        if (recv_line(ctrl, line, sizeof line) <= 0) {
            fprintf(stderr, "control connection lost\n");
            exit(1);
        }
        printf("<- %s\n", line);
        if (first[0] == '\0') memcpy(first, line, 3);  /* codul liniei 1 */
        if (strncmp(line, first, 3) == 0 && line[3] == ' ')
            return atoi(first);                        /* "ddd " îl încheie */
    }
}
```

```
/* Trimite o comandă terminată cu CR LF (parola nu este afișată),
   apoi citește și întoarce codul de răspuns. */
static int command(const char *fmt, ...) {
    char cmd[512];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(cmd, sizeof cmd - 2, fmt, ap);
    va_end(ap);
    printf("-> %s\n", strncmp(cmd, "PASS ", 5) == 0 ? "PASS ****" : cmd);
    strcat(cmd, "\r\n");
    if (send_all(ctrl, cmd, strlen(cmd)) < 0) {
        fprintf(stderr, "send failed: %d\n", WSAGetLastError());
        exit(1);
    }
    return reply();
}

int main(int argc, char *argv[]) {
    if (argc < 6) {
        fprintf(stderr, "usage: %s host port user password ls [dir]"
                        " | get file\n", argv[0]);
        return 1;
    }
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) return 1;
    int get = strcmp(argv[5], "get") == 0;
    if ((ctrl = tcp_connect(argv[1], argv[2])) == INVALID_SOCKET) {
        fprintf(stderr, "cannot connect: %d\n", WSAGetLastError());
        return 1;
    }
    if (reply() != 220) return 1;                     /* serviciu pregătit */
    if (command("USER %s", argv[3]) == 331 &&
        command("PASS %s", argv[4]) != 230) return 1;  /* autentificat? */
    command(get ? "TYPE I" : "TYPE A");          /* tip imagine sau ASCII */

    /* PASV: răspunsul este 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2) */
    printf("-> PASV\n");
    send_all(ctrl, "PASV\r\n", 6);
    char line[1024];
    int h1, h2, h3, h4, p1, p2;
    do {
        if (recv_line(ctrl, line, sizeof line) <= 0) return 1;
        printf("<- %s\n", line);
    } while (strncmp(line, "227 ", 4) != 0);
    char *q = strchr(line, '(');
    if (!q || sscanf(q, "(%d,%d,%d,%d,%d,%d)",
                     &h1, &h2, &h3, &h4, &p1, &p2) != 6) return 1;
    char port[8];
    snprintf(port, sizeof port, "%d", p1 * 256 + p2);
    printf("   data connection: %d.%d.%d.%d port %d*256+%d = %s\n",
           h1, h2, h3, h4, p1, p2, port);
    SOCKET data = tcp_connect(argv[1], port); /* același host, portul din PASV */
    if (data == INVALID_SOCKET) {
        fprintf(stderr, "cannot open data connection\n");
        return 1;
    }

    int code = get ? command("RETR %s", argv[6])
             : argc > 6 ? command("LIST %s", argv[6]) : command("LIST");
    if (code != 125 && code != 150) {            /* de ex. 550 fișier inexistent */
        closesocket(data);
        command("QUIT");
        return 1;
    }
    FILE *out = get ? fopen(argv[6], "wb") : stdout;
```

```
    if (!out) { perror("fopen"); return 1; }
    char buf[4096];
    int n;
    long total = 0;
    while ((n = recv(data, buf, sizeof buf, 0)) > 0) {
        fwrite(buf, 1, (size_t)n, out);
        total += n;
    }
    closesocket(data);        /* modul flux: sfârșit de fișier = conexiune închisă */
    if (get) {
        fclose(out);
        printf("   %ld bytes written to %s\n", total, argv[6]);
    }
    reply();                                        /* 226 transfer încheiat */
    command("QUIT");                                /* 221 la revedere */
    closesocket(ctrl);
    WSACleanup();
    return 0;
}

# sau: cl windows\ftp_client_win.c ws2_32.lib
gcc -Wall -O2 windows\ftp_client_win.c -o ftp_client_win.exe -lws2_32
.\ftp_client_win.exe 127.0.0.1 2121 student secret ls
.\ftp_client_win.exe 127.0.0.1 2121 student secret get web_server.py
fc.exe /b web_server.py $HOME\ftproot\web_server.py
```

- **Q13.** Cum recunoaște `reply()` sfârșitul unui răspuns pe mai multe linii? Adăugați `command("FEAT")` după autentificare și arătați rezultatul.
- **Q14.** De ce este deschisă conexiunea de date înainte de RETR, iar răspunsul final 226 este citit doar după ce conexiunea de date se închide?
- **Q15.** Schimbați PASV în EPSV și analizați răspunsul `229 (|||port|)`. De ce funcționează EPSV cu IPv6 și prin NAT?
- **Q16.** Adăugați `put file` (TYPE I, PASV, STOR, trimitere, închidere, citirea lui 226); porniți serverul cu `-w`. Arătați încărcarea și o comparație cu `fc.exe /b`.

Aceeași sesiune în Python

`py python\ftp_client.py 127.0.0.1 2121 student secret ls` rulează versiunea cu `ftplib` prezentată în fișa Linux; cu depanarea activată, afișează același dialog ca și clientul dumneavoastră în C.

Dacă ceva nu funcționează

Simptom	Cauza probabilă și ce să încercați
`'py' is not recognized`	Python nu este instalat sau nu se află în PATH; reinstalați de pe python.org cu „Add to PATH” sau folosiți `python`
`curl` afișează un obiect în loc de o pagină	ați tastat aliasul PowerShell: tastați întotdeauna `curl.exe`
ftp.exe: `425 Can't open data connection`	Windows Firewall a blocat conexiunea de date de intrare: permiteți ftp.exe pe rețelele private sau folosiți curl.exe (pasiv)
`AuthenticateAsClient` aruncă o excepție	numele nu se potrivește cu certificatul sau rețeaua blochează portul 443; verificați numele transmis
Wireshark nu arată trafic 127.0.0.1	capturați pe *Adapter for loopback traffic capture*, nu pe Wi-Fi sau Ethernet
`Permission denied (publickey)`	cheia nu a fost adăugată pe server sau ~/.ssh are acolo permisiuni greșite (`chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys`)

Săptămâna viitoare

Cursul 5 completează nivelul aplicație: poșta electronică (SMTP, MIME, POP3 și IMAP), știrile Usenet, sistemul de nume de domenii (DNS) și translatarea adreselor de rețea (NAT). În Laboratorul 5 trimiteți e-mail tastând SMTP, îl citiți cu POP3, construiți interogări DNS octet cu octet peste socket-uri UDP — ordinea octeților în rețea, în practică — și construiți propriul router NAT.